Automated Software Engineering (2026) 33:26
https://doi.org/10.1007/s10515-025-00549-x

Assessing the effectiveness of recent closed-source large language models in fault localization and automated program repair

Bo Wang[1,2] · **Ming Deng**[1,2] · **Mingda Chen**[1,2] · **Youfang Lin**[1,2] · **Jianyi Zhou**[3] · **Jie M. Zhang**[4]

Received: 29 January 2025 / Accepted: 30 July 2025 / Published online: 28 October 2025
© The Author(s), under exclusive licence to Springer Science+Business Media, LLC, part of Springer Nature 2025

Abstract

Large Language Models (LLMs) have made significant advancements in code-related tasks. In the field of automated debugging, fault localization (FL) and automated program repair (APR) are two prevalent topics attracting significant research effort. Recently, in the field of FL and APR, many novel LLM-based approaches have emerged. However, most existing LLM-based studies primarily focus on the GPT models from OpenAI or open-source LLMs. With the rapid development of LLMs, various internet giants have introduced new closed-source models. In addition, due to policy restrictions, some regions can only access the commercial LLMs provided by specified companies. Despite the LLMs of OpenAI, the effectiveness of the other closed-source LLMs in FL and APR remains unknown. To better understand the effectiveness of contemporary closed-source models, we conduct a large-scale empirical study on their performance with respect to FL and APR. Specifically, our study involves 4 recent commercial closed-source LLMs (i.e., GPT-4o-Mini, Ernie-3.5, Qwen-turbo, and Doubao-pro) and 1 open-source LLM (i.e., DeepSeek-V3-chat). Note that only the GPT models have region restrictions among all LLMs we studied. We designed a total of 12 distinct prompt templates, 6 each for FL and APR, incorporating various formats and information sources. We conducted experiments to evaluate the effectiveness of FL and APR on 1036 real Java bugs from two datasets, Defects4J 2.0 and ConDefects. The key findings of the experiments indicate that (1) different LLMs tend to succeed on different sets of bugs in both FL and APR, with relatively little overlap among successful cases, implying the models possess distinct strengths in handling specific kinds of bugs, (2) the effectiveness of prompt templates varies across different models, and (3) the effectiveness of FL and APR capabilities of the studied models is significantly correlated with the bug type. We summarized all 14 findings obtained into 3 implications, which could help researchers further improve the performance of LLMs on FL and APR.

Extended author information available on the last page of the article

Keywords Large language models · Software debugging · Fault localization · Automated program repair · Empirical study

1 Introduction

Bugs are pervasive in modern software systems, posing significant risks to various aspects of daily life. Debugging these issues is a notoriously time-consuming and challenging task, often consuming more than half of a developer's programming time (Gazzola et al. 2018). Therefore, substantial research efforts have focused on developing automated debugging techniques to mitigate this burden for programmers.

Two prominent research areas within the field of automated debugging are Fault Localization (FL) and Automated Program Repair (APR) (Chen et al. 2025; Wong et al. 2016; Martinez et al. 2017; Zhang et al. 2023). FL techniques are designed to automatically identify and localize buggy code elements, such as lines or methods (Lou et al. 2020; Meng et al. 2022; Lou et al. 2021; Xie et al. 2022), while APR techniques aim to automatically generate patches to repair these bugs without human intervention (Zhang et al. 2023; Zhu et al. 2023; Ye et al. 2022a, b; Lutellier et al. 2020). Both FL and APR have increasingly leveraged deep-learning models, which have been instrumental in capturing and understanding the context of buggy code (Zhang et al. 2023; Meng et al. 2022; Lou et al. 2021; Xie et al. 2022). Despite their advancements, the performance of these approaches remains limited. For example, although their training data could include billions of historical bugs or patches, their data scale is still far from large enough (Chen et al. 2025). Additionally, their models still have certain shortcomings in understanding code structure and semantics, such as being unable to handle long-range dependencies when representing code.

In recent years, the remarkable success of Large Language Models (LLMs) has spurred interest in applying these models to FL (Kang et al. 2024; Yang et al. 2024) and APR (Fan et al. 2023; Xia and Zhang 2024; Fan et al. 2023; Xia et al. 2023). Existing LLM-based FL and APR studies almost leverage the LLMs provided by OpenAI (such as the family of ChatGPT and CodeX) or open-source LLMs (such as CodeT5 Wang et al. 2021 and CodeLlama Roziere et al. 2023). For example, the most recent peer-reviewed paper by Xia and Zhang, which was published in 2024, still employs ChatGPT-3.5 (Xia and Zhang 2024) to perform conversational program repair.

With the rapid advancement of large language models (LLMs), companies and research institutions are continuously introducing new closed-source models. Notably, to date, only commercial closed-source models from the OpenAI family (such as Codex and GPT models) have been extensively studied in the context of FL and APR. In contrast, the new closed-source LLMs from other technical companies have received limited attention in the academic community. Furthermore, due to policy restrictions, many internet companies impose regional limitations on the use of their models, resulting in significant variations in model accessibility and performance across different regions. Although we could opt to use the best-performing models, such as those from the GPT series, for SE tasks, it remains critical to evaluate other closed-source models, especially since models from different regions may offer differ-

ent capabilities due to access restrictions. Exploring the effectiveness of these models and comparing them with contemporary open-source LLMs is crucial for providing a comprehensive guide to LLM users and for shaping future research directions. These considerations have motivated us to conduct this study, aiming to fill existing gaps in the literature and offer a more thorough understanding for LLM users.

In this paper, we intend to evaluate the effectiveness of recent closed-source general-purpose LLMs in FL and APR. We conducted a large-scale empirical study on 1036 real-world Java bugs from 2 bug datasets, where 760 bugs from Defects4J 2.0 (Just et al. 2014) and 276 bugs from ConDefects (Wu et al. 2024). Our study covers 5 LLMs, including 4 commercial closed-source LLMs (i.e., GPT-4o-Mini, Ernie-3.5, Qwen-turbo, and Doubao-pro) and 1 open-source LLM (i.e., Deepseek-V3-Chat). We employ the latest open-source LLM to provide a baseline for comparing the performance differences between open-source and closed-source LLMs in FL and APR tasks, offering insights for users to choose the most suitable models. Among these models, GPT-4o-Mini is from OpenAI and is restricted in some regions, while the others have no region restrictions. We design a total of 12 distinct prompt templates, 6 each for FL and APR, by incorporating various formats and information sources. Specifically, we involve various elements, including source code, stack traces, and failure assertions in different prompts, utilizing 2 different formats, each specifying either the target code lines or the surrounding code context. To achieve a direct understanding of the capabilities of LLMs, following existing LLM-based studies (Xia and Zhang 2022; Fan et al. 2023; Xia et al. 2023), we perform *zero-shot setting* by interacting APIs of these LLMs, i.e., performing FL and APR directly generate feedback from LLMs under the non-conversational style, without fine-tuning or training on corpus of bugs.

We conducted the experiments on both FL and APR. The results indicate that all these LLMs could successfully localize 558 (53.9%) and fix 181 (17.5%) out of all bugs. We further analyzed the results and gained 14 key findings. These findings can be summarized as follows.

1. Although Deepseek-V3 and GPT-4o-Mini achieved the highest overall performance, the other models exhibit low overlap in successfully located/repaired bugs, indicating varied capabilities across different models. For FL, Deepseek-V3 and GPT-4o-Mini successfully locate 365 and 345 out of 1036 bugs, respectively, while the remaining LLMs locate 126 (i.e., 84 Defects4J bugs and 42 ConDefects bugs) bugs that are uncovered by them. For APR, Deepseek-V3 and GPT-4o-Mini correctly fix 149 and 115 bugs, respectively, while the remaining LLMs supply 19 more bugs (i.e., 11 Defects4J bugs and 8 ConDefects bugs).
2. For complex code-related tasks such as FL and APR, prompt design plays a crucial role. For ChatGPT 3.5, the trend is clear that the prompts with more information exhibit better performance in FL. However, for another model, adding more information may decrease the FL performance. Our analysis of different prompts for FL and APR reveals that each task has a distinct prompt that performs significantly better than others (i.e., *FL P5* and *APR P2*). This suggests that when designing prompts, providing concise yet essential contextual information while specifying a clear and simple output format can effectively enhance the

performance of LLMs. For instance, all the prompts specified to directly return code snippets significantly outperform the prompts specified to encapsulate the results with a JSON format, while for FL, the prompts with stack traces but without test assertions outperform others.
3. We further analyze the correlation between bug types and the bugs successfully localized or fixed by LLMs, confirming a significant relationship between them. Most bugs located or fixed by LLMs are simple bugs within a single line or even a single expression. While the bugs of complex bug types challenge the effectiveness of current LLMs.

Based on the findings, we summarized 3 implications that can help developers better utilize closed-source LLMs for FL and APR, while also providing researchers with valuable insights for proposing new methods in the future.

All the code and experimental results are publicly accessible in our repository: https://github.com/Ming-3364/Study-on-LLMs-for-FL-and-APR.

In summary, our paper makes the following contributions:

- We perform a large-scale empirical study on FL and APR, considering the recent closed-source LLMs that are rarely explored. We studied 5 LLMs on 1036 real-world Java bugs from 2 datasets.
- We evaluate different prompt templates for FL and APR, respectively. For each task, we design 6 templates considering the context and output format, and evaluate their effectiveness on each model.
- We first analyze the correlations between the bug types and the FL and APR effectiveness of LLMs.
- We observed 14 key findings from the experiment results and summarized 3 implications for guiding future research and practice use for LLM-based FL and APR.

The remainder of this paper is organized as follows. In Section 2, we elaborate on the design of our study. In Section 3, we show the evaluation results and the findings analyzed from the results. In Seciton 4, we discuss the implications of our study and the threats to validity. In Section 5, we present the related work on LLM for code tasks, FL, and APR approaches. In Section 6, we conclude this paper.

2 Study design

In this section, we illustrate the design of our study.

2.1 Overview

Figure 1 illustrates the overview of our study. For each bug from the selected benchmark, we create prompts by customizing a predefined template with context information extracted from the code and test failure messages. We designed distinct prompt templates specifically for FL and APR. These tailored prompts are then sent to the

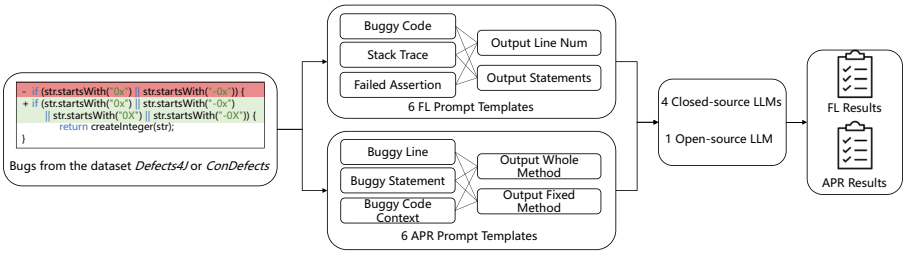


Fig. 1 An overview of our study

target LLMs via invoking their APIs, and their responses are collected and processed. If the prompt length exceeds the token limit of the LLMs, we truncate it accordingly to fit within the constraints. For the FL results generated by the LLMs, we automatically evaluate whether the identified lines or statements fall within the actual bug locations. For APR results, we automatically apply the generated patches and execute them against the whole test suite. If a patch successfully passes all test cases, we further conduct a manual analysis to determine whether it is semantically equivalent to the developer's patch. Note that these evaluation settings align with established practices in both research fields.

2.2 Research question

We investigate the performance of selected LLMs in FL and APR by answering the following research questions (RQs).

- **RQ1: How do different closed-source LLMs perform in FL?**
- **RQ2: How do different prompts affect the effectiveness of closed-source LLMs in FL?**
- **RQ3: How do different bug types affect the effectiveness of closed-source LLMs in FL?**
- **RQ4: How do different closed-source LLMs perform in APR?**
- **RQ5: How do different prompts affect the effectiveness of closed-source LLMs in APR?**
- **RQ6: How do different bug types affect the effectiveness of closed-source LLMs in APR?**

The first three RQs explore their effectiveness for FL, while the last three RQs explore for APR. For both FL and APR, we first evaluate the overall performance of the LLMs we studied, then evaluate the effectiveness of different prompt engineering strategies, and finally investigate the correlation between the bug type and the performance of the LLMs.

2.3 Studied models

Currently, research on the performance of LLMs in coding tasks mainly focuses on open-source models (e.g., CodeLlama or CodeBert) or LLMs of OpenAI (e.g., the

Table 1 The chosen LLMs in our study

Model	Type	Company	Released Date
ChatGPT-4o-Mini	Closed-source	OpenAI	July, 2024
Ernie-3.5	Closed-source	Baidu	December, 2024
Qwen-turbo	Closed-source	Alibaba	December, 2024
Doubao-pro	Closed-source	ByteDance	May, 2024
Deepseek-V3	Open-source	Deepseek	December, 2024

GPT models). Our objective is to extend this exploration by evaluating LLMs beyond those covered in existing studies. As explained before, due to regional restrictions driven by policy, we specifically target closed-source commercial LLMs released in China. Moreover, since our study aims to assess the debugging capabilities of LLMs, the selected models must be proficient in handling coding tasks. Therefore, we focus on the most widely used models in China, namely Ernie, Qwen, and Doubao, which are from top-tier technological enterprises Baidu, Alibaba, and ByteDance, respectively. For comparison with existing studies, in our evaluation, we also employ the recent LLM from OpenAI, GPT-4o-Mini, and the recent high-impact open-source LLM, Deepseek-V3. Deepseek is the latest high-impact open-source LLM (Guo et al. 2025), which is employed as an important baseline for comparison with closed-source models. All the models we studied claim that they support coding-related tasks and provide API access. For each model, we select the most recent version to evaluate its current capacities in FL and APR. The details of the selected models are shown as Table 1.

2.4 Datasets

We intend to evaluate the debugging capabilities of LLMs on real bugs. We employ the widely studied benchmark Defects4J v2.0 (Just et al. 2014), a dataset containing 760 real-world Java bugs from 17 high-impact open-source projects. The bugs of Defects4J are collected from the development procedure of these projects. Each bug from Defects4J contains a buggy version extracted from the development history, a fixed version which is the developers' patch for the very bug, and at least one failed test to trigger the bug. The bugs from Defects4J 2.0 are considered source-code-related, reproducible, and isolated (Sobreira et al. 2018). Defects4J is considered to be the baseline benchmark in the field of FL (Li et al. 2019; Lou et al. 2021; Benton et al. 2020; Zou et al. 2019; Zeng et al. 2022; Meng et al. 2022; Yang et al. 2024) and APR (Xin and Reiss 2017; Xiong et al. 2017; Xin and Reiss 2017; Wen et al. 2018; Jiang et al. 2018; Ye et al. 2022a; Xiao et al. 2024; Xiong and Wang 2022; Ye et al. 2022b; Zhu et al. 2023; Wang et al. 2024; Xin et al. 2024; Xia and Zhang 2024).

To mitigate the risk of overfitting by the LLMs on widely used benchmark datasets, we utilized an alternative dataset, ConDefects (Wu et al. 2024), which is derived from programming competition problems hosted on AtCoder[1] along with their corresponding solutions. From the ConDefects dataset, we select a total of 276 bugs from the most recent dates for our study.

[1] https://atcoder.jp/

Table 2 The real-world bugs used in our study

Dataset	Project	# of Bugs
Defects4J	Chart	23
Defects4J	Cli	37
Defects4J	Closure	171
Defects4J	Codec	15
Defects4J	Collections	2
Defects4J	Compress	42
Defects4J	Csv	14
Defects4J	Gson	17
Defects4J	JacksonCore	24
Defects4J	JacksonDatabind	100
Defects4J	JacksonXml	5
Defects4J	Jsoup	84
Defects4J	JxPath	20
Defects4J	Lang	60
Defects4J	Math	92
Defects4J	Mockito	35
Defects4J	Time	19
ConDefects	-	276
Total	-	1036

Overall, we evaluate three LLMs on both FL and APR against all 1036 real-world Java bugs. Table 2 illustrates the projects and their bugs.

2.5 FL experiments setups

In our study, we directly use a zero-shot fashion single-round conversation to perform FL. Directly querying an LLM for answers remains a widely used baseline approach in research and also in development practice. Note that although the multi-round conversation typically improves the performance, we adopt the one-round setting to evaluate the LLMs in a simpler, more straightforward context. This allows us to eliminate the potential confounding effects of complex interactions on the results. The study of one-round interactions provides a benchmark for future multi-round studies, helping to more clearly identify and quantify the benefits of multi-round interactions. Furthermore, single-round conversations still have practical significance, especially in resource-constrained or time-sensitive environments.

For each bug from Defects4J and ConDefects, we extract the surrounding context and incorporate it into predefined prompt templates, then feed the prompts to LLM and check whether the localization results.

If the bug is located within a method, we use the method containing the bug as the context. If the bug is outside of a method (e.g., in a field), we include the class's field section as the context. For bugs located within a single method but spanning multiple hunks (i.e., multi-hunk bugs), we consider localization successful if at least one of the hunks is correctly identified. For bugs distributed across multiple methods, all relevant methods must be correctly identified for the localization to be considered successful. This setup follows the conventions used in existing FL studies (Zou et al. 2019; Pearson et al. 2017). Moreover, since current LLMs have limitations on the

length of the context they can keep in memory, we cannot include excessively long code in a single-round conversation. This setup also aligns with the practical constraints of LLMs.

2.5.1 Evaluation metrics

For each bug, we classify the localization results into the following three categories:

- **Correct:** The model accurately identifies the statement containing the bug.
- **Incorrect:** The model identifies a statement, but it is not the real buggy statement.
- **Failed:** The model is unable to complete the localization due to errors in understanding or other issues.

Note that the number of all bugs equals the sum of the bugs that are labeled by *Correct*, *Incorrect*, and *Failed*.

As aforementioned, in this study, we use a zero-shot fashion FL approach, where the LLM directly returns the statement it believes is most likely to contain the bug in a single-round conversation. Our setup is thus equivalent to calculating *Top-1* in traditional fault localization (FL). We count the number of *Correct* bugs as our *Top-1* score, which follows the existing study of LLM-based FL (Yang et al. 2024). Additionally, we aim to minimize the number of *Incorrect* and *Failed* results. Among these, *Incorrect* results are particularly harmful because they could confuse developers and would cost more time for debugging. In other words, when the number of *Correct* localization results is the same, fewer *Incorrect* localization results indicate better performance.

2.5.2 Prompts for FL

Since our fault localization (FL) process is conducted in a zero-shot, single-round conversation fashion, we directly instruct the LLMs to identify and return the buggy statement. Effective prompt engineering requires consideration of various information sources, as highlighted in prior studies (Guo et al. 2024; Yang et al. 2024; Wang et al. 2024). Specifically, we incorporate three key types of information: source method code (i.e., `src`), stack trace (i.e., `stack`), and bug-triggering assertion code (i.e., `assert`).

- *Source method code* of the target method provides the static context of the program, enabling the model to analyze the structure and semantics of the code.
- *Stack traces* offer dynamic runtime information extracted from test executions, which can help pinpoint the location of the bug by revealing the execution flow at the time of failure.
- Bug-triggering assertion code represents the specific conditions under which the bug manifests, providing additional context for the model to identify the faulty behavior.

Furthermore, the format in which the model outputs its response can significantly impact the results. To explore this aspect, we instruct the LLMs to generate outputs in two distinct formats:

- Returning the line number of the buggy statement.
- Returning the statement itself.

These elements are systematically combined into different prompt templates, as outlined below. By varying the combination of information sources and output formats, we aim to evaluate their influence on the model's performance in fault localization tasks. This approach allows us to better understand how different prompt designs can leverage the strengths of LLMs in understanding code structure, semantics, and runtime behavior.

- **FL Prompt 1:** "Source code: " + `src` + "There is a bug in the above code, please help me locate it. Please output *only one integer*: the line number of the buggy code (the code starts from line 1). Do not provide any explanations or additional content."
- **FL Prompt 2:** "Source Code: " + `src` + "Stack trace: " + `stack` + "There is a bug in the above code, please help me locate it by considering the stack trace. Please output *only one integer*: the line number of the buggy code (the code starts from line 1). Do not provide any explanations or additional content."
- **FL Prompt 3:** "Source Code: " + `src` + "Stack trace: " + `stack` + "Failure assertion code: " + `assert` + "There is a bug in the above code, please help me locate it by considering the stack trace information and failure assertion code. Please output *only one integer*: the line number of the buggy code (the code starts from line 1). Do not provide any explanations or additional content."
- **FL Prompt 4:** "Source code: " + `src` + "There is a bug in the above code, please help me locate it. Output only the exact *buggy statement*, with no line numbers, explanations, or additional content. Your response should contain just the buggy code line. Do not include "‘ java or "‘ markers in the response."
- **FL Prompt 5:** "Source Code: " + `src` + "Stack trace: " + `stack` + "There is a bug in the above code, please help me locate it. Output only the exact *buggy statement*, with no line numbers, explanations, or additional content. Your response should contain just the buggy code line. Do not include "‘ java or "‘ markers in the response."
- **FL Prompt 6:** "Source Code: " + `src` + "Stack trace: " + `stack` + "Failure assertion code: " + `assert` + "There is a bug in the above code, please help me locate it. Output only the exact *buggy statement*, with no line numbers, explanations, or additional content. Your response should contain just the buggy code line. Do not include "‘ java or "‘ markers in the response."

Following the experience of the existing study (Yang et al. 2024), we use the layout placing the instructions at the tail of prompts, which helps LLMs better understand and remember the requirement of the task. Additionally, we require the LLMs

to avoid the illegal output formats that are frequently generated. By comparing the results of these prompts, we can evaluate how the dynamic information and test code affect the FL performance. Note that the ConDefects dataset has no stack trace or assertion, so we only evaluate the FL capabilities of these LLMs by prompts 1 and 4 against this dataset.

2.6 APR experiments setups

Following the existing study that leverages the LLMs for APR, we adopt the zero-shot fashion (Xia and Zhang 2022; Xia et al. 2023; Fan et al. 2023). This querying mode is still important in research and practice. More specifically, given a buggy code fragment, we instruct the LLMs to return the correct version of the whole fragment directly. We follow existing LLM-based APR approaches, which perform repair given by perfect fault localization (Xia et al. 2023; Xia and Zhang 2022, 2024), i.e., the buggy lines or statements are specified. Same as the setting of the FL experiments, we also adopt the single-round conversation by the similar reason. Evaluating the effectiveness of a single-round conversation can be used as an important baseline. Therefore, we believe the model providing a correct patch in a single response is of practical value. To this end, we only consider the first result returned by LLM, which is similar to the *Top-1* metric used in FL.

2.6.1 Evaluation metrics

Similar to the setup of FL, we also classify the results returned by LLM into several categories by their distance to the correct patch.

- **Correct:** The patch given by LLM is semantically equal to the one of the developers.
- **Incorrect:** The patch given by LLMs is incorrect. An Incorrect patch may lead to compiler errors, failed tests, or overfitting. Note that an *overfitting* patch passes all tests, but it does not behave as the developers intend (Qi et al. 2015; Xiong et al. 2017).
- **Failed:** The model can not generate a patch, due to truncation on response or misunderstanding the given task.

We automatically extract the patches from the response of LLMs and apply them to the corresponding buggy program, then compile and run the patched program. If the patch enables all tests to pass, we manually check whether it is semantically correct, which follows the common practice of APR (Zhang et al. 2023). Since tests do not provide a complete specification of a program's correctness, we have to manually compare the semantics of the generated patches to those of the developers' patches. We only consider the case of *Correct* as a successful repair.

2.6.2 Prompts for APR

For the task of Automated Program Repair (APR), we follow the APR strategies via LLMs established in existing studies (Fan et al. 2023; Xia et al. 2023). We assume that the buggy code fragment has been correctly located, and the buggy code along with its surrounding context is provided to the model. This approach aligns with common practices in LLM-based APR (Fan et al. 2023; Xia et al. 2023), as it ensures that the model has sufficient contextual information to generate accurate fixes. More specifically, we instruct the LLMs to perform repairs in three distinct ways:

- *"Fix line X of the given code"*: This targets a specific line number, allowing the model to focus on localized changes.
- *"Fix the statement S of the given code"*: This directs the model to repair a specific statement, which may span multiple lines.
- *"Fix the last line of the given code": This tests the model's ability to handle edge cases where the bug is located at the end of the code fragment.*

For the output formats of APR, we explore two options:

- *Returning the whole fixed method*: This provides a complete view of the repaired code, ensuring that the model considers the overall structure and semantics.
- *Returning the patched lines specified by line numbers*: This focuses on the minimal changes required to fix the bug, which is useful for understanding the model's precision.

The prompt templates are as follows.

- **APR Prompt 1:** `src` + "There is a bug in line X of the code, please help me fix it. Please return the *complete fixed method*. Do not skip or omit any part of the code. Do not include "‘ java or "‘ markers in the response."
- **APR Prompt 2:** `src` + "There is a bug in S, please help me fix it. Please return the *complete fixed method*. Do not skip or omit any part of the code. Do not include "‘ java or "‘ markers in the response."
- **APR Prompt 3:** `src` + "There is a bug in the last statement, please help me fix it. Please return the *complete fixed method*. Do not skip or omit any part of the code. Do not include "‘ java or "‘ markers in the response."
- **APR Prompt 4:** `src` + "There is a bug in line X of the code, please help me fix it. Please return the results in a strict JSON format as described below. Please follow these instructions carefully: ..."
- **APR Prompt 5:** `src` + "There is a bug in S, please help me fix it. Please return the results in a strict JSON format as described below. Please follow these instructions carefully: ..."
- **APR Prompt 6:** `src` + "There is a bug in the last statement, please help me fix it. Please return the results in a strict JSON format as described below. Please follow these instructions carefully: ..."

Table 3 Bug types of the bugs in Defects4J

ID	Bug Type	ID	Bug Type
1	Conditional Block	6	Missing Null-Check
2	Expression Fix	7	Copy/Paste
3	Wraps-with	8	Constant Change
4	Single Line	9	Code Moving
5	Wrong Reference	10	Other

Note that for the APR prompts 4, 5, and 6, the output specification at their tails is omitted, which can be found in the repo of our study. The study of Fan et al. (2023) indicates that LLMs of the OpenAI family perform differently when specified by the line number or by the buggy code line, which inspires us to design the prompt templates 1, 2, 4, and 5. The study of Xia et al. (2023) indicates that different LLMs are sensitive to various types of context, so we designed prompt templates 3 and 6 with only the previous context before the bug.

2.7 Bug types

We have observed that in existing work on using LLMs for bug fixing and localization, even the best-performing methods still fall short of practical applicability. For instance, in the latest study by Xia and Zhang (2024), after applying various prompt-engineering techniques, ChatGPT was able to fix only about 28.9% of bugs in Defects4J v1.2, leaving a significant number of bugs unresolved. Therefore, we aim to further explore the relationship between defect types and the FL and bug-fixing capabilities of LLMs, providing insights that may inspire future research in automated debugging.

We adopt the widely used bug category by Sobreira et al. (2018), which classifies the bugs of Defects4J 1.2 into 10 bug types, shown as Table 3. The frequency of the bug type is ordered via *ID*, i.e., *Conditional Block* is the most frequently occurring while *Others* is the least. The authors label the bug types and are available at the repository[2]. Since a single bug can belong to multiple categories, we retain the most specific root cause as its primary label. Note that the provided labels are available only for Defects4J v1.2 (Sobreira et al. 2018), which contains 395 bugs, all of which are included in Defects4J v2.0. Therefore, our bug type analysis focuses solely on the bugs from Defects4J v1.2. For each bug successfully localized or fixed by the LLMs, we analyze which types of bugs the current LLMs are more proficient at handling.

The descriptions of the bug types are as follows.

1. *Conditional Block*: The root cause of bugs of this type is the absence of conditional blocks, which may involve missing specific conditions needed to return or throw a particular exception.
2. *Expression Fix*: The bugs of this type are caused by wrong logic expressions, which may involve modifying, narrowing, or widening the conditions.

[2] https://program-repair.org/defects4j-dissection/

3. *Wraps-with*: This type of bug arises from overlooked conditions. To fix such bugs, the faulty code needs to be wrapped with a conditional branch.
4. *Single Line*: This type of bug is only with one line addition, deletion, or modification.
5. *Wrong Reference*: This type of bug is caused by incorrectly referencing a variable or a method call.
6. *Missing Null-Check*: This type of bug is caused by missing the conditions with null-checks.
7. *Copy-Paste*: This type of bug occurs when multiple locations in the code should be modified consistently, but some of the necessary changes are missed.
8. *Constant Change*: This type of bug occurs when using the wrong constant values.
9. *Code Moving*: This type of bug occurs when the code lines are misplaced or positioned incorrectly within the program.
10. *Other*: The bugs that do not fit into the previously defined types.

3 Evaluation

In this section, we illustrate the results and our analysis.

3.1 RQ1: FL performances of the studied LLMs

For a better understanding of the overall performance of the studied LLMs on FL, we merge all the bugs successfully located in all FL prompt templates. Figure 2 shows the overall FL results of all studied LLMs. To better understand their per-

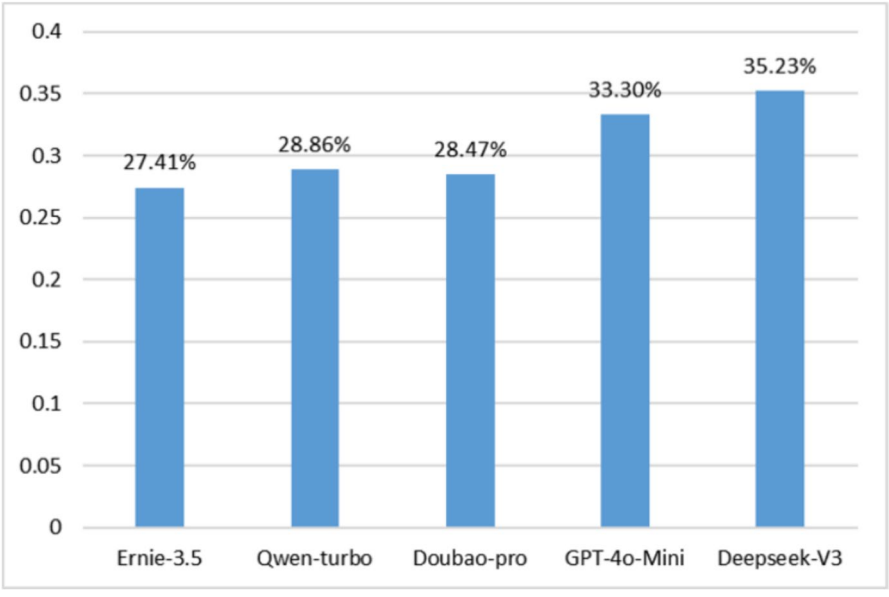


Fig. 2 The percentage of successfully localized bugs for each LLM

Table 4 The number of successfully localized bugs via a single-round conversation

Project	Bug #	Ernie-3.5	Qwen	Doubao	GPT-4o-Mini	Deepseek
Chart	23	**9**	**9**	6	5	7
Cli	37	15	15	12	**19**	**19**
Closure	171	37	41	40	44	**47**
Codec	15	7	7	5	8	**9**
Collections	2	0	0	1	0	**2**
Compress	42	11	13	**18**	17	17
Csv	14	5	7	6	6	**9**
Gson	17	**7**	5	6	6	5
JacksonCore	24	6	4	6	7	**10**
JacksonDatabind	100	24	25	28	30	**33**
JacksonXml	5	1	0	**2**	1	**2**
Jsoup	84	26	35	32	**37**	**37**
JxPath	20	6	**8**	4	6	6
Lang	60	15	19	20	20	**25**
Math	92	29	32	39	39	**41**
Mockito	35	17	**21**	15	19	**21**
Time	19	7	7	6	8	**10**
Defects4J Total	760	222	248	246	272	**300**
ConDefects	276	62	51	49	**73**	65
Total	1036	284	299	295	345	**365**

Note: The best-performing model in each row is highlighted in bold

formance, Table 4 presents the results for each project, and the results of the LLM with the best performance are highlighted in bold.

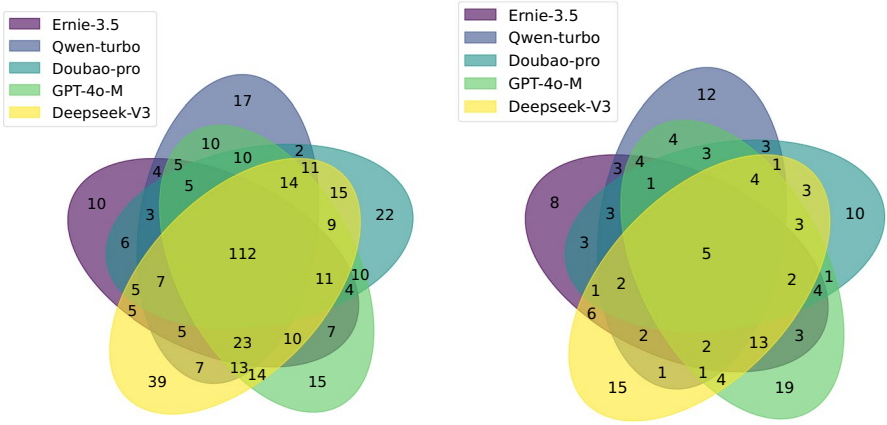
From the figure and the table, we can find that Deepseek-V3 achieves the best FL performance, successfully locating 365 bugs in total (i.e., 35.2% out of 1036 bugs), where 300 bugs (i.e., 39.5% out of 760 bugs) from Defects4J and 65 bugs from ConDefects (i.e., 23.6% out of 276 bugs). In the majority of the projects, Deepseek-V3 is either the best or the second-best effective LLM. It achieves the highest number of successfully localized bugs in several projects, such as Closure (47 bugs), Lang (25 bugs), and Math (41 bugs). Additionally, it ties for the highest performance in projects like Jsoup (37 bugs) and JacksonXml (2 bugs). However, there are a few cases where GPT-4o-M outperforms it, such as in Jsoup (37 bugs, tied), Compress (17 bugs), and JacksonDatabind (30 bugs).

Among the remaining models, GPT-4o-M follows closely behind Deepseek-V3, successfully localizing 345 bugs in total (accounting for 33.3% of all bugs), with 272 bugs from Defects4J (35.8%) and 73 bugs from ConDefects (26.4%). The Qwen-turbo and Doubao-pro models demonstrate comparable performance, with 299 (28.9%) and 295 (28.5%) total successfully localized bugs, respectively. Ernie-3.5 shows the least effective results, localizing 284 bugs in total, corresponding to 27.4% of all cases.

Finding 1 (RQ1): Deepseek-V3 demonstrates the highest success rate in localizing bugs through a single-round conversation, successfully locating 365 out of 1036 bugs (35.2%). GPT-4o-M follows closely, identifying 345 bugs (33.3%), while Qwen-turbo and Doubao-pro achieve similar performance with 299 (28.9%) and 295 (28.5%) successfully localized bugs, respectively. Ernie-3.5 achieves the lowest success rate, locating 284 bugs (27.4%).

Figures 3a and 3b show the Venn diagrams of uniquely located bugs for the LLMs studied on Defects4J and ConDefects, respectively. In the Defects4J dataset, Deepseek-V3 successfully localizes the highest number of unique bugs, totaling 39, followed by Doubao-pro with 22 unique bug localizations. In the ConDefects dataset, GPT-4o-Mini archives the highest number of unique bugs, which is 19, followed by Deepseek-V3 with 15 unique bugs. In total, 167 bugs can be uniquely localized by a specific model, with 103 from the Defects4J dataset and 64 from the ConDefects dataset.

Although Deepseek-V3 demonstrates a significantly superior ability to locate unique bugs compared to the other models, all these LLMs hold a significant number of uniquely located bugs. Specifically, every model has a non-negligible number of unique bugs. For example, even the model with the lowest number of uniquely localized bugs, Ernie-3.5, still contributes 18 bugs (i.e., 10.8% out of all the 167 unique bugs) missed by the others on both datasets. This suggests that these models, when used together, could potentially cover a broader range of bugs than any single model alone. The observed differences in the strengths of these LLMs suggest that selecting appropriate models based on their characteristics could help improve the overall effectiveness of automated debugging tasks, particularly in achieving more comprehensive FL. Meanwhile, all models produce a substantial number of false positives in FL, indicating the need for more effective LLM-based localization methods.



(a) Defects4J (b) ConDefects

Fig. 3 The intersection of successfully localized bugs of the studied LLMs

Finding 2 (RQ1): These models exhibit substantial differences in the bugs they can successfully localize, with each model identifying a distinct subset of faults. Deepseek-V3 demonstrates the strongest capability by localizing 39 unique bugs, which is the highest.

In Section 2.5.1, we classified the FL results into three categories, i.e., *Correct*, *Incorrect*, and *Failed*. We further analyze the distributions of FL results. As aforementioned, ConDefects has neither stack nor assertion, which disables its P2, P3, P5, and P6. Therefore, we only analyze the results on the Defects4J dataset. Table 5 presents the FL results of these LLMs. We can find that only Ernie-3.5 and Doubao-pro generate *Failed* results, with Ernie-3.5 generating more than 500 and Doubao-pro generating 10 *Failed* cases across the first three prompt templates. Analyzing the distribution of *Correct* results, we observe that Deepseek-V3 achieves the highest number of correct localizations overall, particularly excelling in prompt templates P5 and

Table 5 FL results of the LLMs across prompts on Defects4J

Prompt	LLM	Correct	Incorrect	Failed
FL P1	Ernie-3.5	7	45	708
	Qwen-turbo	64	696	0
	Doubao-pro	82	670	8
	GPT-4o-M	65	695	0
	Deepseek-V3	53	707	0
FL P2	Ernie-3.5	2	220	538
	Qwen-turbo	36	724	0
	Doubao-pro	33	721	6
	GPT-4o-M	10	750	0
	Deepseek-V3	21	739	0
FL P3	Ernie-3.5	0	193	567
	Qwen-turbo	17	743	0
	Doubao-pro	32	724	4
	GPT-4o-M	4	756	0
	Deepseek-V3	16	744	0
FL P4	Ernie-3.5	151	609	0
	Qwen-turbo	146	614	0
	Doubao-pro	119	641	0
	GPT-4o-M	156	604	0
	Deepseek-V3	142	618	0
FL P5	Ernie-3.5	155	605	0
	Qwen-turbo	134	626	0
	Doubao-pro	123	637	0
	GPT-4o-M	198	562	0
	Deepseek-V3	205	555	0
FL P6	Ernie-3.5	108	652	0
	Qwen-turbo	82	678	0
	Doubao-pro	54	706	0
	GPT-4o-M	125	635	0
	Deepseek-V3	177	583	0

P6, with 205 and 177 successful localizations, respectively. GPT-4o-M closely follows, achieving 198 correct results in P5 and 156 in P4, showing consistent and reliable performance. Qwen-turbo and Doubao-pro achieve comparable performance in some prompts but fall short in others, indicating their varying effectiveness depending on the prompt type. Regarding *Incorrect* results, most models tend to produce a higher number of incorrect localizations in the first three prompt templates. Qwen-turbo records 743 incorrect results in P3, and GPT-4o-M records 750 in P2, highlighting the challenges faced by LLMs when handling certain prompt structures. Overall, the analysis reveals that the prompt design significantly influences the performance of different models. Prompts P4 and P5 yield the highest number of correct results across all models, suggesting that incorporating richer contextual information can enhance the FL capabilities of LLMs.

Additionally, the current zero-shot fashion FL by LLMs suffers from low precision. For example, even the best performing one, Deepseek-V3 with *FL P5*, only achieves an FL precision of 27.0%. This emphasizes the necessity of developing advanced LLM-based FL approaches.

To further compare the FL results across LLMs, we select the results from the most consistently effective prompt, i.e., *FL P4*. We can find that for Ernie, Qwen, Doubao, GPT-4o-Mini, and Deepseek-V3, the proportions of incorrectly localized bugs are 80%, 82%, 86%, 80%, and 81%, respectively. This indicates that despite differences in overall performance, all models struggle with a high rate of incorrect localizations, which highlights the challenges in accurately localizing buggy statements.

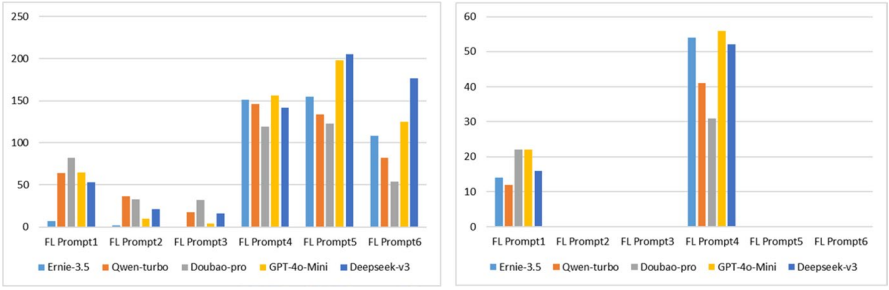
By analyzing the performance trends from *FL P4* to *FL P6*, we can find that most models first increase and then decrease. Note that in terms of information sources in prompts, *P4*<*P5*<*P6*. Therefore, the results indicate that too much information is harmful.

Finding 3 (RQ1): Deepseek-V3 achieves the highest correct localizations, especially with prompts P5 and P6, followed by GPT-4o-M. Ernie-3.5 and Doubao-pro are the only models that produce *Failed* results. Despite the varying performance, all models exhibit high incorrect localization rates.

3.2 RQ2: The impact of different prompts on the FL performance of the studied LLMs

To understand how different prompts impact the FL effectiveness of the studied LLMs, we count the number of successfully located bugs by each prompt template.

Figure 4a presents the number of successfully located bugs across different prompt templates for the Defects4J dataset. Among these prompts, *FL P4* and *FL P5* yield the highest number of correct localizations across all models. Notably, Deepseek-V3 achieves the best performance, particularly excelling in *FL P5* and *FL P6*, where it outperforms the other models. GPT-4o-Mini follows closely, demonstrating competitive performance, especially in *FL P4*. In contrast, Ernie-3.5 struggles in *FL P1*,



(a) Defects4J (b) ConDefects

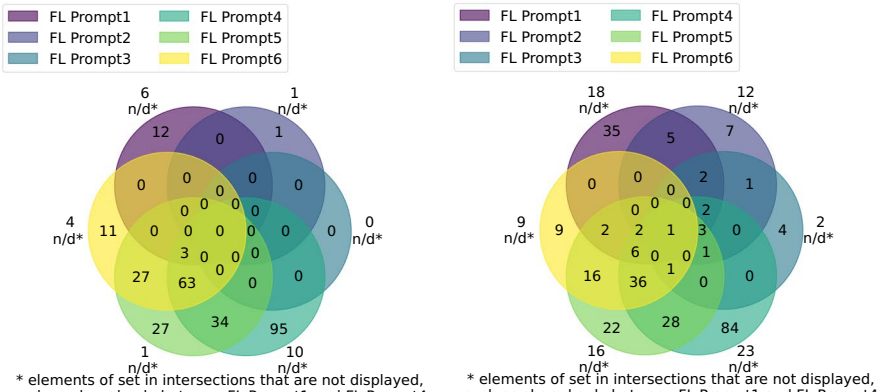
Fig. 4 The successfully located bugs by different prompts

P2, and *P3*, achieving the lowest number of correct localizations. Qwen-turbo and Doubao-pro show varying performance across different prompts.

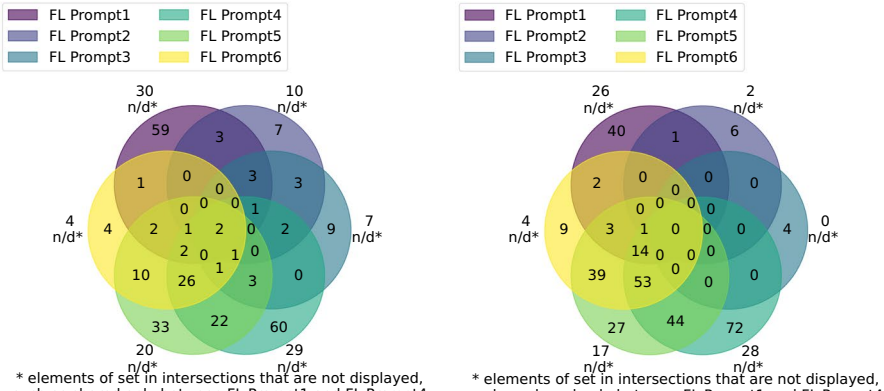
Figure 4b presents the FL performance on the ConDefects dataset. Since ConDefects lacks stack traces and assertion information, only prompts FL P1 and FL P4 are applicable. Deepseek-V3 achieves the highest number of correct localizations in FL P4, while GPT-4o-Mini follows closely with a similar performance. Qwen-turbo and Doubao-pro show relatively balanced performance, but their success rates remain lower compared to Deepseek-V3 and GPT-4o-Mini. Ernie-3.5 still has the lowest number of correct localizations.

Finding 4 (RQ2): Our analysis reveals that prompt design significantly impacts the FL performance of LLMs. Prompts that instruct models to directly output the buggy statements (i.e., *P3, P4, and P5*) significantly outperform those that require returning only the buggy line numbers. Furthermore, an appropriate balance of information, such as relevant code snippets and test outputs, improves performance, while excessive or irrelevant information can lead to decreased effectiveness.

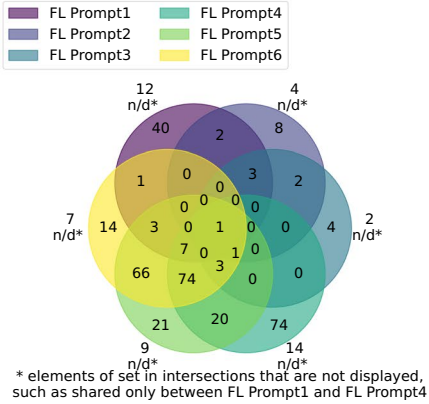
Figure 5 presents the successfully localized bugs by each prompt template (i.e., *FL P1-P6*) of these LLMs, respectively. Overall, most prompts contribute to uniquely successful localized bugs, among which *FL P5* consistently achieves the highest unique bug number across all these LLMs, highlighting its superior effectiveness in FL. For instance, Deepseek-V3 successfully localizes 74 bugs uniquely using FL P5, the highest among all prompts. Similarly, GPT-4o-Mini and Doubao-pro achieve their highest unique bug counts with FL P5, at 72 and 60, respectively. In contrast, prompts *FL P1*, *FL P2*, and *FL P3* generally contribute fewer unique localizations and exhibit more overlap with other prompts. Additionally, the intersection of bugs among prompts indicates that a combination of multiple prompts can improve FL coverage.



(a) Ernie-3.5 (b) Qwen-trubo



(c) Doubao-pro (d) GPT-4o-Mini



(e) Deepseek-V3

Fig. 5 FL results by each prompt

Finding 5 (RQ2): Each prompt contributes uniquely to successful bug localizations, with *FL P5* consistently achieving the highest number of uniquely localized bugs across all LLMs. Conversely, prompts such as *FL P1*, *FL P2*, and *FL P3* demonstrate limited effectiveness. The results indicate that combining multiple prompts can enhance the FL performance of LLMs.

From Table 5, we can compare the FL results across different prompts. The table shows that *FL P1*, which includes basic source code information, performs the worst overall, with Ernie-3.5 and Deepseek-V3 generating the highest number of failed cases (708 and 707, respectively). In contrast, prompts *FL P4* and *FL P5* achieve the highest number of correct localizations across all models, with Deepseek-V3 and GPT-4o-M excelling in *FL P5*, achieving 205 and 198 correct results, respectively. However, no consistent trend is observed for prompts *FL P2* and *FL P3*, as their effectiveness varies across models. While Qwen-turbo and GPT-4o-Mini maintain a high number of incorrect localizations, they do not produce failed results. These findings suggest that excessive or insufficient prompt information can hinder LLM performance, leading to a higher number of incorrect or failed results. Prompts *FL P4* and *FL P5*, which provide a balanced mix of contextual details, yield the best results across all models, highlighting the importance of prompt design in enhancing FL performance.

Finding 6 (RQ2): *FL P2* consistently performs the worst across all LLMs, indicating overload of prompts may mislead the LLM in FL tasks. In contrast, Prompts *FL P4* and *FL P5*, which provide a balanced mix of contextual details, yield the best results across all models.

3.3 RQ3: The impact of bug types on the FL performance of the studied LLMs

Table 6 presents the correlation between the studied LLMs and bug type. The table is organized by bug type and prompt, with percentages indicating the proportion of each bug type in the successfully located bugs. Note that due to the lack of label data on the bugs of Defects4J v2.0, we only analyze the bugs from Defects4J v1.2, which are labeled by the previous study (Sobreira et al. 2018). Since Ernie-3.5 has no successfully localized bugs within Defects4J v1.2 under *FL P1-P2*, the corresponding rows are all zeros. The results are grouped by models (i.e., Ernie-3.5, Qwen-turbo, Doubao-pro, GPT-4o-Mini, and Deepseek-V3) and prompt templates (i.e., *FL P1-P6*).

Across all LLMs, *Conditional Block* bugs demonstrate the highest success rates, with Deepseek-V3 achieving the best performance. Similarly, GPT-4o-Mini shows strong performance in handling *Conditional Block* bugs, with success rates reaching 50% under FL P2. However, more complex bug types, such as *Wrong Reference* and *Missing Null-Check*, are consistently challenging for all models, with success rates remaining below 10%.

Table 6 The correlations between bug type and successful FL results of LLMs

LLM	Prompt	CB	EF	WW	SL	WR	MN	CP	CC	CM	Others
Ernie	**FL P1**	25%	25%	25%	25%	0%	0%	0%	0%	0%	0%
	FL P2	0%	0%	0%	0%	0%	0%	0%	0%	0%	0%
	FL P3	0%	0%	0%	0%	0%	0%	0%	0%	0%	0%
	FL P4	29%	17%	7%	12%	3%	4%	7%	3%	0%	19%
	FL P5	38%	18%	8%	10%	1%	4%	4%	1%	0%	16%
	FL P6	38%	15%	8%	13%	0%	4%	2%	2%	0%	19%
Qwen	**FL P1**	33%	27%	7%	13%	0%	7%	0%	0%	0%	13%
	FL P2	47%	18%	12%	6%	0%	0%	0%	0%	0%	18%
	FL P3	29%	0%	14%	14%	0%	0%	0%	14%	0%	29%
	FL P4	37%	15%	11%	11%	3%	3%	1%	1%	0%	18%
	FL P5	38%	10%	8%	7%	2%	3%	3%	3%	0%	26%
	FL P6	30%	9%	7%	9%	0%	5%	5%	5%	2%	28%
Doubao	**FL P1**	38%	13%	10%	8%	3%	3%	8%	8%	0%	13%
	FL P2	44%	22%	0%	11%	0%	0%	0%	0%	0%	22%
	FL P3	33%	8%	8%	8%	0%	0%	8%	0%	0%	33%
	FL P4	28%	9%	11%	13%	3%	3%	3%	3%	0%	27%
	FL P5	31%	15%	8%	12%	3%	5%	2%	0%	0%	24%
	FL P6	22%	4%	13%	22%	0%	9%	4%	4%	0%	22%
GPT-4o-M	**FL P1**	33%	10%	7%	10%	3%	7%	0%	0%	3%	27%
	FL P2	50%	0%	0%	0%	50%	0%	0%	0%	0%	0%
	FL P3	100%	0%	0%	0%	0%	0%	0%	0%	0%	0%
	FL P4	29%	18%	8%	12%	3%	1%	1%	4%	0%	24%
	FL P5	36%	14%	9%	11%	2%	1%	3%	3%	0%	20%
	FL P6	29%	19%	6%	13%	0%	2%	4%	6%	0%	21%
Deepseek	**FL P1**	68%	9%	5%	5%	0%	0%	0%	5%	0%	9%
	FL P2	60%	20%	10%	0%	0%	0%	0%	0%	10%	0%
	FL P3	50%	0%	25%	25%	0%	0%	0%	0%	0%	0%
	FL P4	30%	22%	8%	12%	1%	4%	1%	3%	0%	18%
	FL P5	28%	20%	8%	9%	2%	4%	5%	3%	1%	19%
	FL P6	31%	23%	7%	8%	1%	2%	6%	3%	1%	18%

Note: **CB** - Conditional Block, **EF** - Expression Fix, **WW** - Wraps-with, **SL** - Single Line, **WR** - Wrong Reference, **MN** - Missing Null-Check, **CP** - Copy/Paste, **CC** - Constant Change, **CM** - Code Moving

The effectiveness of different prompts varies across bug types. Among the prompts, *FL P5* consistently delivers the best performance across all models, highlighting its robustness in helping LLMs identify faults. Deepseek-V3 achieves 28% for *Conditional Block* bugs and 20% for *Expression Fix* bugs using *FL P5*. In contrast, prompts FL P1 and FL P3 show lower effectiveness in handling complex bug types.

The results indicate that specific bug types influence FL outcomes, with CB and SL bugs being easier to localize compared to WR and MN bugs. This suggests that LLMs excel in scenarios with clear structural patterns but struggle with issues requiring deeper semantic understanding.

The FL results suggest some characteristics of the LLMs. On the one hand, the LLMs are particularly adept at capturing certain types of logical errors, such as *Conditional Block* and *Expression Fix*. On the other hand, LLMs tend to be less sensitive to subtle differences, as evidenced by the poor performance of all three models in

handling *Constant Change* bugs, which is related to constant replacement. This characteristic is consistent with the work of Li et al. (2023).

Finding 7 (RQ3): The studied closed-source LLMs, they perform well on the bug types *Conditional Block* and *Expression Fix*, while being limited to localize the *Code Moving* bugs which involve large-scale changes and the *Constant Change* bugs which are subtle syntactical differences.

3.4 RQ4: APR performances of the studied LLMs

To answer RQ4, we conduct the repair experiments via a single-round conversation, zero-shot fashion. Figure 6 presents the overall results, which combine all prompt templates of the studied LLMs on APR. Additionally, Table 7 presents the detailed results for each project. For each project or dataset, the results of the LLM with the best performance are highlighted in bold.

From the figure and the table, we observe that Deepseek-V3 achieves the best overall performance, successfully fixing 83 bugs in the Defects4J dataset (i.e., 10.9% of the total 760 bugs) and 66 bugs in the ConDefects dataset (i.e., 23.9% of the total 276 bugs). GPT-4o-Mini follows closely, fixing 62 bugs in Defects4J (8.2%) and 53 bugs in ConDefects (19.2%). For other models, Ernie-3.5 successfully repairs 35 bugs in Defects4J (4.6%) and 39 bugs in ConDefects (14.1%), while Doubao-pro achieves similar results with 42 fixes in Defects4J and 43 in ConDefects, showing competitive but slightly lower performance. Qwen-turbo demonstrates the lowest repair effectiveness, fixing only 14 bugs in Defects4J and 18 in ConDefects, making it the least effective model among the studied LLMs.

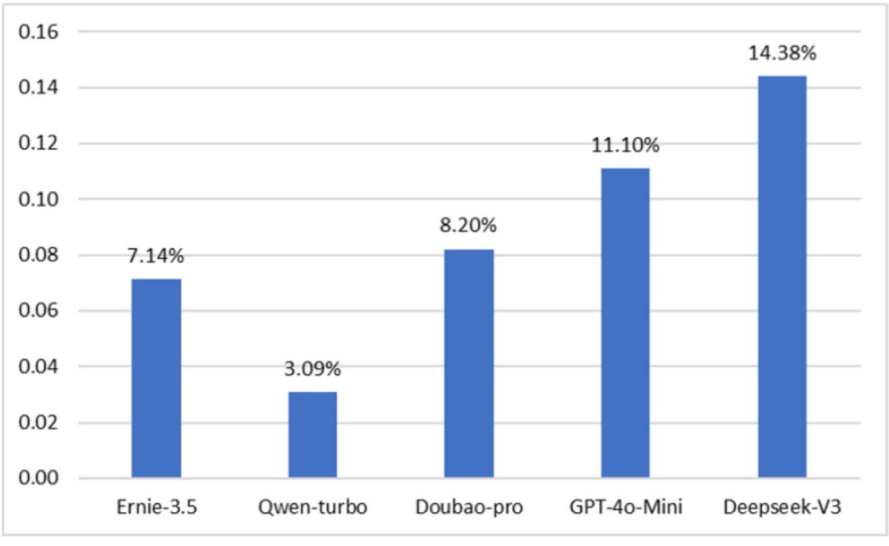


Fig. 6 The percentage of successfully fixed bugs for each LLM

Table 7 The number of successfully fixed bugs via a single-round conversation

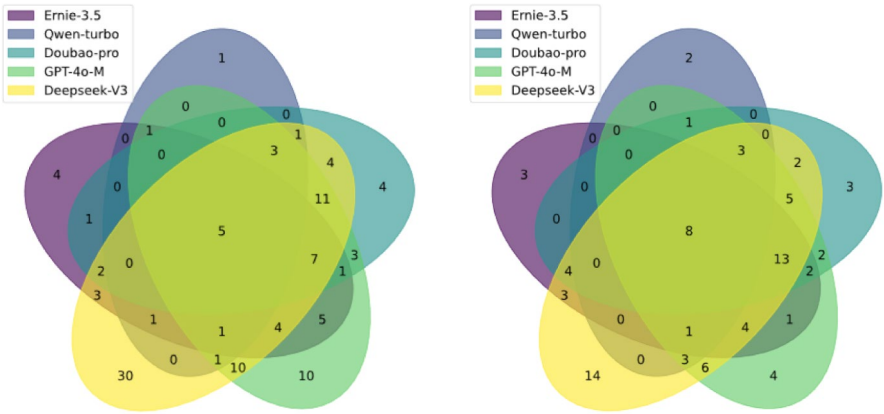
Project	# of Bugs	Ernie-3.5	Qwen	Doubao	GPT-4o-Mini	Deepseek
Chart	23	5	2	6	5	**7**
Cli	37	0	0	1	3	**5**
Closure	171	7	0	3	6	**12**
Codec	15	0	0	**1**	0	**1**
Collections	2	**1**	0	0	0	0
Compress	42	1	0	2	4	**6**
Csv	14	1	1	1	**2**	0
Gson	17	0	0	0	**1**	0
JacksonCore	24	1	2	**4**	**4**	2
JacksonDatabind	100	2	1	6	5	**11**
JacksonXml	5	0	0	0	0	0
Jsoup	84	5	1	5	**9**	**9**
JxPath	20	0	0	0	0	0
Lang	60	6	1	3	7	**8**
Math	92	5	4	7	11	**14**
Mockito	35	1	2	2	3	**6**
Time	19	0	0	1	**2**	**2**
Defects4J Total	760	35	14	42	62	**83**
ConDefects	276	39	18	43	53	**66**
Total	1036	74	32	85	115	**149**

Note: The best-performing model in each row is highlighted in bold

Overall, Deepseek stands out as the most effective model, successfully fixing a total of 149 bugs across both datasets, followed by GPT-4o-Mini with 115 bugs, while Ernie-3.5 and Doubao achieve moderate results with 74 and 85 fixed bugs, respectively. Qwen lags behind with only 32 total fixes. The results highlight the superior performance of Deepseek in APR tasks across both datasets, indicating its strong ability to generate effective fixes in a single-round conversation.

Finding 8 (RQ4): For the APR task conducted in a zero-shot, single-round conversation setting, Deepseek-V3 demonstrates the highest success rate, correctly fixing 149 out of 1036 bugs (i.e., 14.4%). GPT-4o-Mini follows with 115 successfully fixed bugs (i.e., 11.1%).

Figures 7a and 7b show the Venn diagrams of uniquely fixed bugs for the three LLMs studied on Defects4J and ConDefects, respectively. In total, 113 bugs from Defects4J and 104 bugs from ConDefects are successfully fixed by all models. Among all the 217 bugs fixed, 75 bugs (34.6%) can only be fixed by one unique model, comprising 49 from Defects4J and 26 from ConDefects. For both datasets, Deepseek-V3 fixes the most unique bugs, with 30 from Defects4J and 14 from ConDefects, showing a significant advantage over other models. GPT-4o-Mini closely follows with a total of 14 unique fixes, comprising 10 from Defects4J and 4 from ConDefects. Doubao-pro fixes 4 and 3, Ernie-3.5 fixes 4 and 3, and Qwen-turbo fixes 1 and 2 unique



(a) Defects4J (b) ConDefects

Fig. 7 The intersection of successfully fixed bugs of the studied LLMs

bugs in Defects4J and ConDefects, respectively. Every model contributes unique fixes to both datasets, suggesting that combining multiple models could enhance the overall effectiveness of APR.

> **Finding 9 (RQ4):** Among all 217 successfully fixed bugs, 75 (34.6%) can only be uniquely fixed by one model. Deepseek-V3 contributes 44 uniquely fixed bugs, achieving the best performance. This indicates that different models tend to fix different bugs, revealing substantial variation in their repair capabilities.

As shown in Section 2.6.1, a patch returned by APR techniques can be classified into three categories, i.e., *Correct*, *Incorrect*, and *Failed*. For the patches that pass all test cases, we further manually analyze whether it is *Correct*, which is a common practice in APR (Jiang et al. 2018; Wen et al. 2018; Xin and Reiss 2017; Xiong and Wang 2022). Table 8 presents the count of different APR result types of the three LLMs.

Overall, Deepseek-V3 achieves the highest success rates, particularly excelling in *APR P2*, where it correctly fixes 62 bugs in Defects4J and 34 in ConDefects, followed by GPT-4o-Mini, which successfully repairs 48 and 42 bugs, respectively. *APR P2* consistently yields the best performance across all models. In contrast, APR P3 performs the worst, with most models failing to generate any correct patches. Failure rates vary significantly across prompts, with *APR P5* and *P6* exhibiting the highest number of failed cases, particularly for Doubao-pro and Ernie-3.5, where Doubao-pro fails in 45 cases under APR P6. While Deepseek-V3 and GPT-4o-Mini demonstrate the strongest repair capabilities, Qwen-turbo struggles the most, failing to fix any bugs under certain prompts (e.g., APR P1 and APR P3 in ConDefects).

Table 8 The count of different APR results of the LLMs

Dataset		Defects4J			ConDefects		
Prompt	LLM	Correct	Incorrect	Failed	Correct	Incorrect	Failed
APR P1	Ernie-3.5	12	748	0	22	254	0
	Qwen-turbo	0	760	0	0	276	0
	Doubao-pro	19	741	0	25	251	0
	GPT-4o-M	35	725	0	27	249	0
	Deepseek-V3	33	727	0	14	262	0
APR P2	Ernie-3.5	22	738	0	22	254	0
	Qwen-turbo	1	759	0	0	276	0
	Doubao-pro	34	726	0	33	243	0
	GPT-4o-M	48	711	0	42	234	0
	Deepseek-V3	62	698	0	34	242	0
APR P3	Ernie-3.5	0	760	0	2	274	0
	Qwen-turbo	0	760	0	0	276	0
	Doubao-pro	4	756	0	5	271	0
	GPT-4o-M	1	759	0	1	275	0
	Deepseek-V3	0	760	0	1	275	0
APR P4	Ernie-3.5	9	735	16	18	251	7
	Qwen-turbo	8	745	7	13	262	1
	Doubao-pro	13	721	26	13	259	4
	GPT-4o-M	11	746	3	21	255	0
	Deepseek-V3	30	729	1	39	236	1
APR P5	Ernie-3.5	8	738	14	4	263	9
	Qwen-turbo	6	747	7	4	272	0
	Doubao-pro	4	724	32	4	268	4
	GPT-4o-M	6	747	7	9	263	4
	Deepseek-V3	10	750	0	12	264	0
APR P6	Ernie-3.5	3	731	26	8	251	17
	Qwen-turbo	2	752	6	1	275	0
	Doubao-pro	4	711	45	4	261	11
	GPT-4o-M	2	748	10	5	267	4
	Deepseek-V3	6	751	3	15	260	1

These findings highlight that prompt selection plays a critical role in APR performance, with APR P2 proving the most effective.

To further understand the precision of repair, we analyze the *Incorrect* patches, and identify the ones that pass all test cases, (i.e., overfitting patches). Following the common practice of FL and APR, we calculate the precision and recall of each LLM across all prompts, shown as Table 9. The table shows that the LLMs with a higher number of correct patches have relatively high precision. For example, Deepseek-V3 under *APR P4* achieves the highest number of correct fixes with a precision of 0.711.

Additionally, as will be further discussed in the next research question, *APR P2* achieves the best performance in terms of the number of correctly fixed bugs. To better understand its effectiveness, we analyze the categories of patches generated by *APR P2* across different models, as shown in Table 10. Each patch is classified based on its evaluation outcome: *Pass All Tests*, *Fail Some Tests*, *Timeout*, or *Compile Failed*.

Table 9 The precision and recall of APR

Prompt	LLM	Correct	Overfitting	Incorrect	Precision	Recall
APR P1	**Ernie-3.5**	34	15	1002	0.694	0.033
	Qwen-turbo	0	1	1036	0.000	0.000
	Doubao-pro	44	11	992	0.800	0.042
	GPT-4o-M	62	32	974	0.660	0.060
	Deepseek-V3	47	21	989	0.691	0.045
APR P2	**Ernie-3.5**	44	17	992	0.721	0.042
	Qwen-turbo	1	2	1035	0.333	0.001
	Doubao-pro	67	9	969	0.882	0.065
	GPT-4o-M	90	40	945	0.692	0.087
	Deepseek-V3	96	41	940	0.701	0.093
APR P3	**Ernie-3.5**	2	1	1034	0.667	0.002
	Qwen-turbo	0	0	1036	0.000	0.000
	Doubao-pro	9	5	1027	0.643	0.009
	GPT-4o-M	2	3	1034	0.400	0.002
	Deepseek-V3	1	0	1035	1.000	0.001
APR P4	**Ernie-3.5**	27	17	986	0.614	0.026
	Qwen-turbo	21	17	1007	0.553	0.020
	Doubao-pro	26	12	980	0.684	0.025
	GPT-4o-M	32	20	1001	0.615	0.031
	Deepseek-V3	69	28	965	0.711	0.067
APR P5	**Ernie-3.5**	12	5	1001	0.706	0.012
	Qwen-turbo	10	3	1019	0.769	0.010
	Doubao-pro	8	2	992	0.800	0.008
	GPT-4o-M	15	6	1010	0.714	0.014
	Deepseek-V3	22	7	1014	0.759	0.021
APR P6	**Ernie-3.5**	11	8	982	0.579	0.011
	Qwen-turbo	3	0	1027	1.000	0.003
	Doubao-pro	8	4	972	0.667	0.008
	GPT-4o-M	7	6	1015	0.538	0.007
	Deepseek-V3	21	8	1011	0.724	0.020

Table 10 The patch evaluation results of *APR P2* across LLMs

	Pass All Tests	Fail Some Tests	Timeout	Compile Failed
Ernie-3.5	61 (5.9%)	192 (18.5%)	2 (0.2%)	781 (75.4%)
Qwen-turbo	3 (0.3%)	4 (0.4%)	0 (0%)	1029 (99.3%)
Doubao-pro	76 (7.3%)	679 (65.5%)	10 (1.0%)	271 (26.2%)
GPT-4o-M	130 (12.5%)	614 (59.3%)	9 (0.9%)	283 (27.3%)
Deepseek-V3	137 (13.2%)	381 (36.8%)	8 (0.8%)	510 (49.2%)

Among all the models, Doubao-pro exhibits the lowest rate of compilation failure, with only 26.2% of its generated patches failing to compile. GPT-4o-Mini follows closely, with a compilation failure rate of 27.3%. However, even for the patches that pass compilation, most are rejected by the test suites due to failing at least one test. For instance, the number of *Fail Some Tests* patches generated by Deepseek-V3 is approximately two times larger than the number of its *Pass All Tests* patches. These

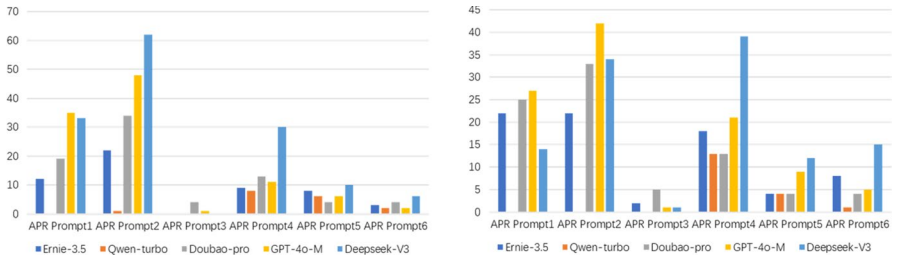
results show the difficulty of generating patches that are both syntactically valid and semantically correct, highlighting a key challenge in LLM-based APR.

Compared to FL, the proportion of false positives in APR (i.e., plausible but incorrect patches) is relatively lower. This suggests that in APR tasks, patches generated by different LLMs are more likely to be either invalid (e.g., fail to compile) or clearly incorrect (e.g., fail tests), making it easier to identify and filter out low-quality results. As a result, combining APR outcomes from different models appears more feasible and reliable than in FL, where false positives are harder to detect. This observation opens up opportunities for future work to explore ensemble or selection mechanisms tailored to LLM-based APR.

Finding 10 (RQ4): Among the studied LLMs, Deepseek-V3 produces the fewest *Failed* results across different prompts, demonstrating its robustness in generating valid patches. However, all models predominantly produce *Incorrect* patches, with APR P2 emerging as the most effective prompt for guiding LLMs in bug fixing. Additionally, complex prompts such as APR P5 and APR P6 lead to higher failure rates, highlighting the challenges of automated program repair and the need for improved prompt design to enhance patch correctness.

3.5 RQ5: The impact of different prompts on the APR performance of the studied LLMs

Figures 8a and 8b show the number of successfully fixed bugs of the studied LLMs by different prompt templates, in Defects4J and ConDefects, respectively. *APR P2* consistently achieves the highest number of successful fixes across all LLMs. For instance, Deepseek-V3 and GPT-4o-Mini achieve their best performance with *APR P2*, fixing 96 and 90 bugs, respectively. *APR P1* also shows relatively strong performance but is slightly less effective than *APR P2*. *APR P3* and *P6* are the least effective, indicating that both the context of before the after the statements to be fixed are



(a) Defects4J (b) ConDefects

Fig. 8 The successfully fixed bugs by different prompts

valuable. By comparing *APR P1-P4*, *APR P2-P5*, and *APR P3-P6*, we can find the output format also has no consistent influence on repair effectiveness.

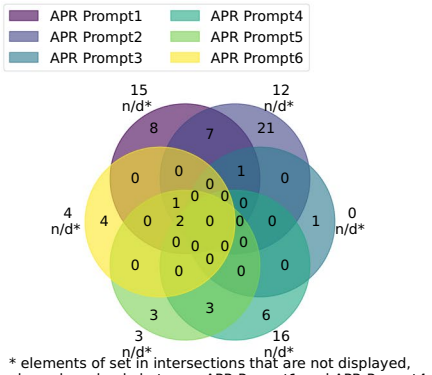
Finding 11 (RQ5): Across both Defects4J and ConDefects, *APR P2* consistently achieves the highest number of successful fixes across all LLMs. *APR P1* also demonstrates strong effectiveness but remains slightly less effective than *APR P2*, while *APR P3* and *APR P6* yield the lowest success rates. Moreover, the output format does not have a consistent impact on repair effectiveness.

Figure 9 illustrate the performance of different APR prompt templates in fixing bugs across the models we studied. Across most models except Qwen, *APR P2* achieves the highest number of unique successfully fixed bugs, indicating its effectiveness in APR once again. *APR P4* and *P1* follow closely, achieving the second-highest unique successfully fixed bugs on several models. *APR P3* exhibits the least performance, with only one unique bug successfully fixed, and that too by the least performing LLM, Ernie-3.5. Overall, each APR prompt holds uniqueness in fixing certain bugs, indicating that these prompt templates hold significant variation in their repair capabilities.

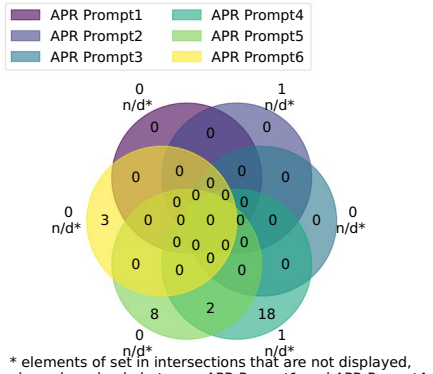
Finding 12 (RQ5): APR P2 consistently achieves the highest number of unique successfully fixed bugs across most LLMs, reinforcing its effectiveness in automated program repair. Despite the varying effectiveness of prompts, each APR prompt contributes uniquely to bug fixing.

Table 8 presents the APR results across different prompt templates. The data reveals that *APR P1*, *P2*, and *P3* produce no *Failed* results across all models, suggesting that contemporary LLMs can effectively understand the instructions and generate complete patched methods. In contrast, *APR P4*, *P5*, and *P6* exhibit a significant proportion of *Failed* results, indicating that more complex output formats hinder LLM performance in APR tasks. Moreover, given the complex output format, the prompt with only half of the code context, *APR P6*, performs the worst.

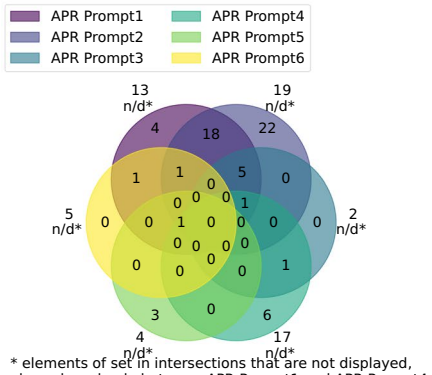
Finding 13 (RQ5): A simple output format improves APR success rates, as seen in *APR P1*, *P2*, and *P3*, which produce no *Failed* results across all models. In contrast, *APR P4*, *P5*, and *P6*, which require more complex output formats, exhibit a significant number of failures.



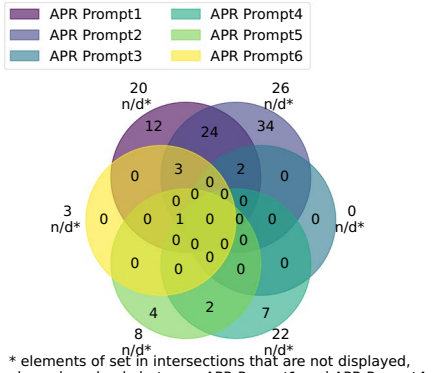
(a) Ernie-3.5



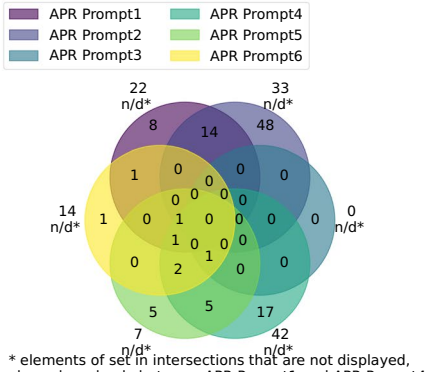
(b) Qwen-trubo



(c) Doubao-pro



(d) GPT-4o-Mini



(e) Deepseek-V3

Fig. 9 APR results by each prompt

3.6 RQ6: The impact of bug types on the APR performance of the studied LLMs

We further analyze the bug types of all successfully fixed bugs, as shown in Table 11. Similar to our analysis in FL, we conduct the correlation analysis only on Defects4J V1.2 bugs, as they are the ones with available bug type labels. Across most models, *Expression Fix* and *Single Line* bugs are the most frequently repaired types. Deepseek-V3 and GPT-4o-Mini achieve the highest success rates in these categories, particularly under *APR P1* and *APR P2*. *Conditional Block* bugs are also well-handled, especially by Deepseek-V3, which achieves 33% success under APR P1 and 19% under APR P2. In contrast, *Wrong Reference* and *Code Moving* bugs are rarely fixed, indicating that LLMs struggle with these categories, possibly due to their reliance on deeper semantic reasoning and broader program context.

Table 11 The correlations between bug type and APR results of LLMs

LLM	Prompt	CB	EF	WW	SL	WR	MN	CP	CC	CM	Others
Ernie	**APR P1**	13%	50%	0%	13%	0%	0%	13%	13%	0%	0%
	APR P2	20%	33%	0%	20%	0%	7%	7%	7%	0%	7%
	APR P3	0%	0%	0%	0%	0%	0%	0%	0%	0%	0%
	APR P4	0%	40%	20%	20%	0%	0%	0%	20%	0%	0%
	APR P5	0%	60%	0%	20%	0%	0%	0%	20%	0%	0%
	APR P6	0%	0%	0%	0%	0%	100%	0%	0%	0%	0%
Qwen	**APR P1**	0%	0%	0%	0%	0%	0%	0%	0%	0%	0%
	APR P2	0%	0%	0%	0%	0%	0%	0%	0%	0%	0%
	APR P3	0%	0%	0%	0%	0%	0%	0%	0%	0%	0%
	APR P4	0%	40%	0%	40%	0%	20%	0%	0%	0%	0%
	APR P5	0%	25%	0%	25%	0%	0%	0%	25%	0%	25%
	APR P6	0%	100%	0%	0%	0%	0%	0%	0%	0%	0%
Doubao	**APR P1**	0%	40%	0%	30%	0%	10%	10%	0%	0%	10%
	APR P2	0%	47%	5%	16%	0%	21%	0%	5%	0%	5%
	APR P3	0%	0%	0%	50%	0%	50%	0%	0%	0%	0%
	APR P4	0%	43%	0%	43%	0%	0%	0%	0%	0%	14%
	APR P5	0%	0%	0%	0%	0%	50%	0%	50%	0%	0%
	APR P6	0%	100%	0%	0%	0%	0%	0%	0%	0%	0%
GPT-4o-M	**APR P1**	11%	33%	6%	22%	0%	11%	6%	6%	0%	6%
	APR P2	4%	42%	4%	19%	0%	15%	4%	4%	0%	8%
	APR P3	0%	0%	0%	0%	0%	100%	0%	0%	0%	0%
	APR P4	0%	60%	0%	20%	0%	0%	0%	0%	0%	20%
	APR P5	0%	40%	0%	20%	0%	0%	0%	20%	0%	20%
	APR P6	0%	0%	0%	0%	0%	100%	0%	0%	0%	0%
Deepseek	**APR P1**	33%	27%	0%	20%	0%	7%	7%	0%	0%	7%
	APR P2	19%	31%	6%	16%	0%	16%	3%	3%	0%	6%
	APR P3	0%	0%	0%	0%	0%	0%	0%	0%	0%	0%
	APR P4	0%	44%	0%	22%	6%	11%	0%	11%	0%	6%
	APR P5	0%	50%	0%	25%	0%	0%	0%	13%	0%	13%
	APR P6	0%	33%	0%	33%	0%	17%	0%	17%	0%	0%

Note: **CB** - Conditional Block, **EF** - Expression Fix, **WW** - Wraps-with, **SL** - Single Line, **WR** - Wrong Reference, **MN** - Missing Null-Check, **CP** - Copy/Paste, **CC** - Constant Change, **CM** - Code Moving

For the prompts, *APR P2* demonstrates the best overall repair success rates across multiple bug types, reaffirming its effectiveness in guiding LLMs to generate correct patches. For the models, Deepseek-V3 consistently outperforms other LLMs across multiple bug types, and GPT-4o-Mini follows closely.

Finding 14 (RQ6): APR performance varies significantly across different bug types. *Expression Fix* and *Single Line* bugs are the easiest to repair, while *Wrong Reference* and *Code Moving* remain the most challenging. Deepseek-V3 and GPT-4o-Mini consistently outperform other LLMs, particularly in handling *Conditional Block* and *Expression Fix* bugs. *APR P2* achieves the highest success rates across multiple bug types.

4 Discussion

In this section, we first discuss the implications of all the findings obtained from Section 3 and then discuss the threats to validity.

4.1 Implications

Based on the previous findings from the experiments, we summarize several implications, which could suggest better practices for using closed-source LLMs and inspire further improvements in proposing novel FL and APR approaches.

4.1.1 Divergence in LLM performance for FL and APR

The first key implication arises from the substantial divergence in the sets of bugs successfully localized or fixed by different LLMs or prompts. As observed in Findings 1–3 and 8–10, each LLM tends to succeed on a different subset of bugs, indicating that their effectiveness varies significantly across fault types and task conditions in both FL and APR. While Deepseek-V3 and GPT-4o-Mini exhibit the best overall performance, the remaining models still demonstrate unique strengths by successfully addressing bugs that the top performers fail to handle. For example, although Deepseek-V3 and GPT-4o-Mini together localized 365 bugs in FL, 126 bugs were successfully localized only by the other two models. In APR, all models suffer from a high proportion of *Failed* patches under the complex output format, but the failure modes differ notably across models. These results highlight the importance of analyzing the characteristics of bugs that each model handles well or poorly. A better understanding of the conditions under which specific LLMs succeed can inform targeted model selection strategies, which could improve the reliability of automated debugging in practical settings.

4.1.2 Characterizing prompt design for different LLMs.

The second key implication we learned is for designing prompts for a specified mission and a specified LLM.

Based on Findings 4 and 8, we discovered that adding more information to the prompt does not always lead to better performance for some models. Therefore, it is essential to design concise prompts tailored to the specific characteristics of each model, rather than merely feeding on more information. We can also observe that prompts requiring the LLM to return the line number of a faulty statement consistently result in worse performance than those asking it to return the code snippet itself. This phenomenon may be related to the capability of LLMs in counting line numbers and how they are trained. We should consider this factor in designing prompts.

Findings 5-6 and 12-13 further reveal that the effectiveness of different prompts varies significantly across LLMs. This suggests that in FL and APR, developing distinct and task-specified prompt strategies for each model is crucial. Additionally, since different prompts resulted in unique successes in both FL and APR, an approach that integrates the results from various prompts could enhance the overall performance.

Moreover, our study reveals that zero-shot, single-round conversation prompting for FL and APR still leaves room for improvement. While this approach provides a baseline for evaluating direct model performance, it is unclear whether multi-round conversations, RAG, or other advanced techniques could substantially enhance the repair and localization accuracy.

4.1.3 Novel FL and APR approach for complex bug types.

The third implication we learned is that the LLMs tend to handle some bug types more sophisticatedly.

By Findings 7 and 14, we observed that the capability of LLMs to localize and repair different types of bugs varies significantly. LLMs perform better in localizing and fixing more localized bugs, such as *Expression Fix* and *Single Line*, but struggle with more complex bugs like *Code Moving*, which involves relocating large blocks of code. This insight has implications for the future design of LLM-based localization and repair methods.

To address the challenges LLMs face with complex bug types like *Code Moving*, several improvements can be made. (1) It may be beneficial to specialize LLMs for handling specific types of bugs, allowing for more precise models tailored to categories such as *Expression Fixes*. (2) Hybrid approaches combining LLMs with traditional program analysis tools could be a solution for more complex tasks like *Code Moving*, where structural analysis could complement LLMs' semantic understanding. (3) Targeted training could improve LLMs' ability to manage complex bug fixes by exposing them to multi-step refactoring tasks. (4) LLMs could be enhanced to retain larger contexts, allowing them to process multi-line or multi-file dependencies. These strategies have the potential to improve the handling of complex bugs in LLM-based FL and APR.

4.2 Threats to validity

In this section, we discuss the potential threats to the validity of our study.

4.2.1 Internal threats

One internal threat to validity arises from the code implementation of our study. To mitigate this risk, we conducted the analysis with great care and open all our implementation and results for policy check. Additionally, for all the APR patches that pass all tests, we manually validate their correctness (i.e., whether they are overfitting patches). To mitigate this risk, for all these patches, one author labeled the results, and another sampled the labeled for validation.

Another internal threat arises from the data leakage issues, where the buggy and fixed versions of the dataset Defects4J (Just et al. 2014) have been introduced into the training data of the LLMs we studied, leading to unreliable FL and APR results. To mitigate this risk, we employ another dataset ConDefects (Wu et al. 2024) that avoids this problem.

4.2.2 External threats

We evaluate three recent closed-source LLMs on two datasets, by three prompt templates for one debugging task, which is of enough scale for observing findings from the experiments. However, our results may still not generalize well to other LLMs and other types of programming languages.

4.2.3 Construct threats

The main construct threat arises from the evaluation metrics we used, e.g., the number of *Top-1* located bugs in FL and the number of *Top-1* fixed bugs in APR. Nevertheless, both metrics are widely adopted in FL and APR communities, and play as the main metric in evaluation.

5 Related work

In this section, we introduce the related work of this study, including LLM for code-related tasks, FL, and APR appraoches.

5.1 LLM for code tasks

LLMs have demonstrated significant potential across a wide range of coding tasks (Wang et al. 2024; Fan et al. 2023). The introduction of the Transformer model (Vaswani et al. 2017) has greatly expanded the scale at which deep learning models can be trained, paving the way for the development of LLMs. These models operate based on natural language instructions, known as *prompts*. Users can interact

with LLMs by providing prompts that may consist of both natural language and code, engaging in conversations that are constrained by the fixed context length of LLM.

The versatility of LLMs across various coding tasks has been widely explored in recent research. LLMs have shown promise in facilitating code generation (Khojah et al. 2024; Mu et al. 2024), where they can autonomously generate code snippets based on high-level descriptions. Additionally, LLMs are increasingly being used for test generation (Yang et al. 2024; Ryan et al. 2024; Schäfer et al. 2023; Yuan et al. 2024), helping to automate the creation of test cases that ensure code reliability and correctness. Furthermore, LLMs have been applied in code analysis tasks (Mu et al. 2024; Endres et al. 2024; Wang et al. 2024) and comment generation (Shin et al. 2023; Mu et al. 2023), where they assist in understanding, refactoring, and optimizing existing code. Among the various applications of LLMs, LLM-enhanced debugging has emerged as a particularly promising area of research, encompassing Fault Localization (FL) and Automated Program Repair (APR), which potentially alleviates developers from the heavy burden of software maintenance.

Most of the studied LLMs are either from the facility of OpenAI (e.g., ChatGPT and CodeX), or open source models (e.g., CodeT5 Wang et al. 2021, CodeLlama Roziere et al. 2023, and DeepSeekCoder Guo et al. (2024) 2024). However, various internet giants are continually releasing new models, particularly commercial closed-source models. The capabilities of these models in handling coding tasks have not been thoroughly studied. Therefore, our study aims to fill this gap by evaluating some of these newly released closed-source models. Specifically, due to policy restrictions, some regions are unable to access OpenAI's LLMs. Therefore, we have deliberately selected models that are accessible in these restricted regions.

5.2 Fault localization

FL is a critical task in software debugging, aimed at identifying the exact locations of faults within a program (Wong et al. 2016; Chen et al. 2025). Over the years, various techniques have been developed to improve the accuracy and efficiency of FL, ranging from traditional approaches like Spectrum-Based Fault Localization (SBFL) and Mutation-Based Fault Localization (MBFL) to more advanced machine learning and deep learning methodologies.

5.2.1 Traditional FL approaches

Early work in FL predominantly focused on statistical methods, particularly SBFL and MBFL. SBFL approaches rely on program spectra—information about which parts of the code are executed during passing and failing test cases—to assign suspiciousness scores to code elements, such as Ochiai (Abreu et al. 2006), Tarantula (Jones and Harrold 2005), and DStar (Wong et al. 2013). MBFL employs the result of mutation analysis to compute the suspicious score of code elements, such as MUSE (Moon et al. 2014), Metallaxis (Papadakis and Le Traon 2015), and BLMu (Wang et al. 2025).

5.2.2 Machine learning-based FL approaches

Machine learning techniques have been increasingly applied to enhance the accuracy of FL. Early approaches focused on statistical machine learning methods, such as learning-to-rank techniques (Xuan and Monperrus 2014; Zou et al. 2019). These methods aimed to improve the precision of FL by leveraging historical bug data and combining suspicious scores from various SBFL approaches. Additionally, models like PageRank (Zhang et al. 2017, 2019) and SmartFL (Zeng et al. 20220 employed probabilistic modeling to further refine FL results.

In recent years, deep learning has emerged as a powerful tool for FL, with researchers developing models that utilize neural networks to analyze program behavior and locate bugs. Approaches such as DeepFL (Li et al. 2019), which incorporates Multilayer Perceptron and Recurrent Neural Networks, and Grace (Lou et al. 2021), which represents program entities via graphs, have shown significant promise. Other methods have explored the use of CNNs for predicting coverage matrices (Li et al. 2021), and graph neural networks for two-phase FL (Li et al. 2022a). Recent advancements include the development of FixLocator (Li et al. 2022b), which targets co-change fixing defects, and GNet4FL (Qian et al. 2023), which combines static and dynamic features to train graph convolutional models. These deep learning-based approaches demonstrate the potential of advanced neural network architectures to improve the precision and effectiveness of FL. Most recently, researchers have employed LLMs for FL (Kang et al. 2024; Yang et al. 2024; Qin et al. 2024). However, these studies only covered the models from the GPT family. For example, AgentFL (Qin et al. 2024) uses multi-agent equipped with ChatGPT for resolving project-level localization.

5.3 Automated program repair

APR holds the promise of fixing buggy programs without human intervention, and it has garnered significant attention in the last decade (Gazzola et al. 2018; Monperrus 2018; Zhang et al. 2023; Chen et al. 2025). Generally, APR techniques primarily refer to the Generate and Validate (G&V) repair methods, where patches are generated and then validated by running tests. If a patch passes all tests, it is considered correct. GenProg (Le Goues et al. 2011) was a pioneering work in G&V APR, and since then, repair techniques have garnered significant attention from the software engineering community. Based on the evolution of APR techniques over the past decade, we roughly categorize them into traditional APR, deep learning-based APR, and large model-based APR.

5.3.1 Traditional APR

In the development of APR, approaches before deep learning-based techniques are categorized as traditional APR. The earliest approach is search-based APR, represented by GenProg (Le Goues et al. 2011), which typically relies on genetic algorithms (Le Goues et al. 2011; Yuan and Banzhaf 2018; Wang et al. 2022, 2024) or random search (Qi et al. 2014) to generate repair patches. Then, researchers proposed template-based repair techniques, where predefined bug-fixing templates are applied

to generate patches, such as PAR (Kim et al. 2013), Anti-Pattern (Tan et al. 2016), and TBar (Liu et al. 2019). Enumeration-based repair systematically generates and evaluates potential patches by exhaustively exploring a predefined search space, such as Kali (Qi et al. 2015) and IBugFinder (Xin et al. 2024). Later, some researchers leveraged statistical methods to generate patches, including Genesis (Long et al. 2017), ACS (Xiong et al. 2017), SimFix (Jiang et al. 2018), CapGen (Wen et al. 2018), ssFix (Xin and Reiss 2017), and Hanabi (Xiong and Wang 2022). Another important class of traditional APR methods is semantics-based APR, where program semantics are translated into constraints and solved using solvers to generate patches, such as SemFix (Nguyen et al. 2013), Nopol (Xuan et al. 2016; Martinez et al. 2017), JAID (Chen et al. 2017), Angelix (Mechtaev et al. 2016), DirectFix (Mechtaev et al. 2015), and ExtractFix (Gao et al. 2021).

5.3.2 Deep learning-based APR

With the increasing availability of large open-source code repositories, deep learning-based APR techniques, which apply self-trained deep learning models to address APR, have gained significant attention. For more detailed information, please refer to the survey of Zhang et al. (2023).

DLFix is the early deep-learning-based APR (Li et al. 2020), which employs a tree-structured RNN to learn from bug fixes and their surrounding AST contexts. CoCoNuT (Lutellier et al. 2020) transforms the APR problem to the neural machine translation (NMT) problem, and proposes a novel context-aware NMT model. RewardRepair (Ye et al. 2022a) involves test execution information inside its model. SelfAPR (Ye et al. 2022b) changes the learning paradigm from supervised training to self-supervised. Recoder (Zhu et al. 2021) implements a novel encoder-decoder architecture to ensure syntactically correct patch generation. KNOD (Jiang et al. 2023) proposes a novel three-stage tree decoder and a domain-rule distillation for patch synthesis. Tare (Zhu et al. 2023) introduces a type-aware neural model designed for program repair, learning typing rules to enhance the effectiveness of the patch generation process.

5.3.3 LLM-based APR

Most recently, with the emerging LLM techniques, numerous APR approaches leveraging LLMs have been proposed.

Xia et al. (2023) and Fan et al. (2023) first leveraged LLMs to generate patches. Zhang et al. proposed GAMMA, which leverages an LLM to predict the correct code for masked code as a fill-in-the-blank task (Zhang et al. 2023). Huang et al. conducted a large-scale empirical study on the effectiveness of fine-tuning open-source LLMs for APR (Huang et al. 2023). Xia et al. proposed FitRepair (Xia et al. 2023), an LLM-based APR approach enhanced with fine-tuning strategies. Xia et al. used a multi-turn dialogue mechanism to enable GPT to generate better patches (Xia and Zhang 2024). Referring to the ongoing survey of Zhang et al. (2024), the current LLM-based APR approaches never covered the closed-source LLMs that are beyond OpenAI, which motivates our study.

6 Conclusion

In this paper, we present an empirical study evaluating the FL and APR capabilities of recent LLMs, particularly the closed-source models that are uncovered by existing studies, across various prompt templates. We conducted the experiments covering 5 LLMs, 12 prompt templates, and 2 real-world datasets. In particular, Ernie-3.5, Doubao, and Qwen are not covered by existing studies in the research area of automated debugging. Our experiments yielded 14 key findings, which we distilled into 3 practical implications for optimizing the use of LLMs. Additionally, our results offer insights that could guide future research in the fields of FL and APR. In conclusion, different models have their unique capabilities for handling some types of bugs, and should be invoked by well-designed prompts.

Acknowledgements We are grateful to the editors and the anonymous reviewers.

Author contributions Bo Wang wrote the main manuscript text. Ming Deng and Mingda Chen conducted the experiments. Youfang Lin, Jianyi Zhou, and Jie M. Zhang revised the manuscript .

Funding This work is supported by the National Natural Science Foundation of China under Grant No. 62202040.

Data availability All the implementation and evaluation results are avaiable at our repo: https://github.com/Ming-3364/Study-on-LLMs-for-FL-and-APR

Declarations

Competing interests The authors declare no competing interests.

References

Abreu, R., Zoeteweij, P., Van Gemund, A.J.: An evaluation of similarity coefficients for software fault localization. In: 2006 12th Pacific Rim International Symposium on Dependable Computing (PRDC'06), pp. 39–46 (2006). IEEE

Benton, S., Li, X., Lou, Y., Zhang, L.: On the effectiveness of unified debugging: An extensive study on 16 program repair systems. In: Proceedings of the 35th IEEE/ACM International Conference on Automated Software Engineering, pp. 907–918 (2020)

Chen, X., Hu, X., Huang, Y., Jiang, H., Ji, W., Jiang, Y., Jiang, Y., Liu, B., Liu, H., Li, X., Lian, X., Meng, G., Peng, X., Sun, H., Shi, L., Wang, B., Wang, C., Wang, J., Wang, T., Xuan, J., Xia, X., Yang, Y., Yang, Y., Zhang, L., Zhou, Y., Zhang, L.: Deep learning-based software engineering: Progress, challenges, and opportunities. Sci. China Inf. Sci. **68**(1), 111102 (2025)

Chen, L., Pei, Y., Furia, C.A.: Contract-based program repair without the contracts. In: 2017 32nd IEEE/ACM International Conference on Automated Software Engineering (ASE), pp. 637–647 (2017). IEEE

Endres, M., Fakhoury, S., Chakraborty, S., Lahiri, S.K.: Can large language models transform natural language intent into formal method postconditions? Proceed. ACM Softw. Eng. **1**(FSE), 1889–1912 (2024)

Fan, Z., Gao, X., Mirchev, M., Roychoudhury, A., Tan, S.H.: Automated repair of programs from large language models. In: 2023 IEEE/ACM 45th International Conference on Software Engineering (ICSE), pp. 1469–1481 (2023). IEEE

Fan, A., Gokkaya, B., Harman, M., Lyubarskiy, M., Sengupta, S., Yoo, S., Zhang, J.M.: Large language models for software engineering: Survey and open problems. arXiv:2310.03533 (2023)
Gao, X., Wang, B., Duck, G.J., Ji, R., Xiong, Y., Roychoudhury, A.: Beyond tests: Program vulnerability repair via crash constraint extraction. ACM Trans. Softw. Eng. Method. (TOSEM) **30**(2), 1–27 (2021)
Gazzola, L., Micucci, D., Mariani, L.: Automatic software repair: A survey. In: Proceedings of the 40th International Conference on Software Engineering, pp. 1219–1219 (2018)
Guo, Q., Cao, J., Xie, X., Liu, S., Li, X., Chen, B., Peng, X.: Exploring the potential of chatgpt in automated code refinement: An empirical study. In: Proceedings of the 46th IEEE/ACM International Conference on Software Engineering, pp. 1–13 (2024)
Guo, D., Yang, D., Zhang, H., Song, J., Zhang, R., Xu, R., Zhu, Q., Ma, S., Wang, P., Bi, X., et al.: Deepseek-r1: Incentivizing reasoning capability in llms via reinforcement learning. arXiv:2501.12948 (2025)
Guo, D., Zhu, Q., Yang, D., Xie, Z., Dong, K., Zhang, W., Chen, G., Bi, X., Wu, Y., Li, Y., et al.: Deepseek-coder: When the large language model meets programming–the rise of code intelligence. arXiv:2401.14196 (2024)
Huang, K., Meng, X., Zhang, J., Liu, Y., Wang, W., Li, S., Zhang, Y.: An empirical study on fine-tuning large language models of code for automated program repair. In: 2023 38th IEEE/ACM International Conference on Automated Software Engineering (ASE), pp. 1162–1174 (2023). IEEE
Jiang, N., Lutellier, T., Lou, Y., Tan, L., Goldwasser, D., Zhang, X.: Knod: Domain knowledge distilled tree decoder for automated program repair. In: 2023 IEEE/ACM 45th International Conference on Software Engineering (ICSE), pp. 1251–1263 (2023). IEEE
Jiang, J., Xiong, Y., Zhang, H., Gao, Q., Chen, X.: Shaping program repair space with existing patches and similar code. In: Proceedings of the 27th ACM SIGSOFT International Symposium on Software Testing and Analysis, pp. 298–309 (2018)
Jones, J.A., Harrold, M.J.: Empirical evaluation of the tarantula automatic fault-localization technique. In: Proceedings of the 20th IEEE/ACM International Conference on Automated Software Engineering, pp. 273–282 (2005)
Just, R., Jalali, D., Ernst, M.D.: Defects4j: A database of existing faults to enable controlled testing studies for java programs. In: Proceedings of the 2014 International Symposium on Software Testing and Analysis, pp. 437–440 (2014)
Kang, S., An, G., Yoo, S.: A quantitative and qualitative evaluation of llm-based explainable fault localization. Proceed. ACM. Softw. Eng. 1(FSE), 1424–1446 (2024)
Khojah, R., Mohamad, M., Leitner, P., Oliveira Neto, F.G.: Beyond code generation: An observational study of chatgpt usage in software engineering practice. Proceed. ACM Softw. Eng. **1**(FSE), 1819–1840 (2024)
Kim, D., Nam, J., Song, J., Kim, S.: Automatic patch generation learned from human-written patches. In: 2013 35th International Conference on Software Engineering (ICSE), pp. 802–811 (2013). IEEE
Le Goues, C., Nguyen, T., Forrest, S., Weimer, W.: Genprog: A generic method for automatic software repair. IEEE Trans. Softw. Eng. **38**(1), 54–72 (2011)
Li, X., Li, W., Zhang, Y., Zhang, L.: Deepfl: Integrating multiple fault diagnosis dimensions for deep fault localization. In: Proceedings of the 28th ACM SIGSOFT International Symposium on Software Testing and Analysis, pp. 169–180 (2019)
Li, Z., Tang, E., Chen, X., Wang, L., Li, X.: Graph neural network based two-phase fault localization approach. In: Proceedings of the 13th Asia-Pacific Symposium on Internetware, pp. 85–95 (2022)
Li, Y., Wang, S., Nguyen, T.N.: Dlfix: Context-based code transformation learning for automated program repair. In: Proceedings of the ACM/IEEE 42nd International Conference on Software Engineering, pp. 602–614 (2020)
Li, Y., Wang, S., Nguyen, T.N.: Fault localization to detect co-change fixing locations. In: Proceedings of the 30th ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering, pp. 659–671 (2022)
Li, Y., Wang, S., Nguyen, T.: Fault localization with code coverage representation learning. In: 2021 IEEE/ACM 43rd International Conference on Software Engineering (ICSE), pp. 661–673 (2021). IEEE
Li, T.-O., Zong, W., Wang, Y., Tian, H., Wang, Y., Cheung, S.-C., Kramer, J.: Nuances are the key: Unlocking chatgpt to find failure-inducing tests with differential prompting. In: 2023 38th IEEE/ACM International Conference on Automated Software Engineering (ASE), pp. 14–26 (2023). IEEE
Liu, K., Koyuncu, A., Kim, D., Bissyandé, T.F.: Tbar: Revisiting template-based automated program repair. In: Proceedings of the 28th ACM SIGSOFT International Symposium on Software Testing and Analysis, pp. 31–42 (2019)

Long, F., Amidon, P., Rinard, M.: Automatic inference of code transforms for patch generation. In: Proceedings of the 2017 11th Joint Meeting on Foundations of Software Engineering, pp. 727–739 (2017)

Lou, Y., Ghanbari, A., Li, X., Zhang, L., Zhang, H., Hao, D., Zhang, L.: Can automated program repair refine fault localization? a unified debugging approach. In: Proceedings of the 29th ACM SIGSOFT International Symposium on Software Testing and Analysis, pp. 75–87 (2020)

Lou, Y., Zhu, Q., Dong, J., Li, X., Sun, Z., Hao, D., Zhang, L., Zhang, L.: Boosting coverage-based fault localization via graph-based representation learning. In: Proceedings of the 29th ACM Joint Meeting on European Software Engineering Conference and Symposium on the Foundations of Software Engineering, pp. 664–676 (2021)

Lutellier, T., Pham, H.V., Pang, L., Li, Y., Wei, M., Tan, L.: Coconut: combining context-aware neural translation models using ensemble for program repair. In: Proceedings of the 29th ACM SIGSOFT International Symposium on Software Testing and Analysis, pp. 101–114 (2020)

Martinez, M., Durieux, T., Sommerard, R., Xuan, J., Monperrus, M.: Automatic repair of real bugs in java: A large-scale experiment on the defects4j dataset. Empir. Softw. Eng. **22**, 1936–1964 (2017)

Mechtaev, S., Yi, J., Roychoudhury, A.: Angelix: Scalable multiline program patch synthesis via symbolic analysis. In: Proceedings of the 38th International Conference on Software Engineering, pp. 691–701 (2016)

Mechtaev, S., Yi, J., Roychoudhury, A.: Directfix: Looking for simple program repairs. In: 2015 IEEE/ACM 37th IEEE International Conference on Software Engineering, **1**, pp. 448–458 (2015). IEEE

Meng, X., Wang, X., Zhang, H., Sun, H., Liu, X.: Improving fault localization and program repair with deep semantic features and transferred knowledge. In: Proceedings of the 44th International Conference on Software Engineering, pp. 1169–1180 (2022)

Monperrus, M.: Automatic software repair: A bibliography. ACM Comput. Surv. (CSUR) **51**(1), 1–24 (2018)

Moon, S., Kim, Y., Kim, M., Yoo, S.: Ask the mutants: Mutating faulty programs for fault localization. In: 2014 IEEE Seventh International Conference on Software Testing, Verification and Validation, pp. 153–162 (2014). IEEE

Mu, F., Chen, X., Shi, L., Wang, S., Wang, Q.: Developer-intent driven code comment generation. In: 2023 IEEE/ACM 45th International Conference on Software Engineering (ICSE), pp. 768–780 (2023). IEEE

Mu, F., Shi, L., Wang, S., Yu, Z., Zhang, B., Wang, C., Liu, S., Wang, Q.: Clarifygpt: A framework for enhancing llm-based code generation via requirements clarification. Proceed. ACM Softw. Eng. **1**(FSE), 2332–2354 (2024)

Nguyen, H.D.T., Qi, D., Roychoudhury, A., Chandra, S.: Semfix: Program repair via semantic analysis. In: 2013 35th International Conference on Software Engineering (ICSE), pp. 772–781 (2013). IEEE

Papadakis, M., Le Traon, Y.: Metallaxis-fl: mutation-based fault localization. Softw. Test., Verific. Reliab. **25**(5–7), 605–628 (2015)

Pearson, S., Campos, J., Just, R., Fraser, G., Abreu, R., Ernst, M.D., Pang, D., Keller, B.: Evaluating and improving fault localization. In: 2017 IEEE/ACM 39th International Conference on Software Engineering (ICSE), pp. 609–620 (2017). IEEE

Qi, Z., Long, F., Achour, S., Rinard, M.: An analysis of patch plausibility and correctness for generate-and-validate patch generation systems. ISSTA, pp. 24–36 (2015)

Qi, Y., Mao, X., Lei, Y., Dai, Z., Wang, C.: The strength of random search on automated program repair. In: Proceedings of the 36th International Conference on Software Engineering, pp. 254–265 (2014)

Qian, J., Ju, X., Chen, X.: Gnet4fl: effective fault localization via graph convolutional neural network. Automat. Softw. Eng. **30**(2), 16 (2023)

Qin, Y., Wang, S., Lou, Y., Dong, J., Wang, K., Li, X., Mao, X.: Agentfl: Scaling llm-based fault localization to project-level context. arXiv:2403.16362 (2024)

Roziere, B., Gehring, J., Gloeckle, F., Sootla, S., Gat, I., Tan, X.E., Adi, Y., Liu, J., Remez, T., Rapin, J., et al.: Code llama: Open foundation models for code. arXiv:2308.12950 (2023)

Ryan, G., Jain, S., Shang, M., Wang, S., Ma, X., Ramanathan, M.K., Ray, B.: Code-aware prompting: A study of coverage-guided test generation in regression setting using llm. Proceed. ACM Softw. Eng. **1**(FSE), 951–971 (2024)

Schäfer, M., Nadi, S., Eghbali, A., Tip, F.: An empirical evaluation of using large language models for automated unit test generation. IEEE Trans. Softw. Eng. (2023)

Shin, J., Wei, M., Wang, J., Shi, L., Wang, S.: The good, the bad, and the missing: Neural code generation for machine learning tasks. ACM Trans. Softw. Eng. Method. **33**(2), 1–24 (2023)

Sobreira, V., Durieux, T., Madeiral, F., Monperrus, M., Almeida Maia, M.: Dissection of a bug dataset: Anatomy of 395 patches from defects4j. In: 2018 IEEE 25th International Conference on Software Analysis, Evolution and Reengineering (SANER), pp. 130–140 (2018). IEEE

Tan, S.H., Yoshida, H., Prasad, M.R., Roychoudhury, A.: Anti-patterns in search-based program repair. In: Proceedings of the 2016 24th ACM SIGSOFT International Symposium on Foundations of Software Engineering, pp. 727–738 (2016)

Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A.N., Kaiser, Ł., Polosukhin, I.: Attention is all you need. Adv. Neural Inf. Process. Syst. **30** (2017)

Wang, J., Huang, Y., Chen, C., Liu, Z., Wang, S., Wang, Q.: Software testing with large language models: Survey, landscape, and vision. IEEE Trans. Softw. Eng. (2024)

Wang, S., Lin, B., Chen, L., Mao, X.: Divide-and-conquer: Automating code revisions via localization-and-revision. ACM Trans. Softw. Eng. Method. (2024)

Wang, B., Liu, G., Lin, Y., Ren, S., Li, H., Zhang, D.: Enhanced evolutionary automated program repair by finer-granularity ingredients and better search algorithms. In: Proceedings of the 13th Asia-Pacific Symposium on Internetware, pp. 107–116 (2022)

Wang, B., Liu, G., Lin, Y., Ren, S., Li, H., Zhang, D.: Enhanced evolutionary automated program repair by finer-granularity ingredients and better search algorithms. J. Softw: Evol. Process. (2024)

Wang, Y., Wang, W., Joty, S., Hoi, S.C.: Codet5: Identifier-aware unified pre-trained encoder-decoder models for code understanding and generation. arXiv:2109.00859 (2021)

Wang, B., Wei, J., Chen, M., Chen, C., Lin, Y., Zhang, J.M.: A systematic exploration of mutation-based fault localization formulae. Softw. Test. Verif. Reliab. **35**(1), 1905 (2025)

Wen, M., Chen, J., Wu, R., Hao, D., Cheung, S.-C.: Context-aware patch generation for better automated program repair. In: Proceedings of the 40th International Conference on Software Engineering, pp. 1–11 (2018)

Wong, W.E., Debroy, V., Gao, R., Li, Y.: The dstar method for effective software fault localization. IEEE Trans. Reliab. **63**(1), 290–308 (2013)

Wong, W.E., Gao, R., Li, Y., Abreu, R., Wotawa, F.: A survey on software fault localization. IEEE Trans. Softw. Eng. **42**(8), 707–740 (2016)

Wu, Y., Li, Z., Zhang, J.M., Liu, Y.: Condefects: A complementary dataset to address the data leakage concern for llm-based fault localization and program repair. In: Companion Proceedings of the 32nd ACM International Conference on the Foundations of Software Engineering, pp. 642–646 (2024)

Xia, C.S., Ding, Y., Zhang, L.: The plastic surgery hypothesis in the era of large language models. In: Proceedings of the 38th IEEE/ACM International Conference on Automated Software Engineering (2023)

Xia, C.S., Wei, Y., Zhang, L.: Automated program repair in the era of large pre-trained language models. In: Proceedings of the 45th International Conference on Software Engineering (ICSE 2023). Ass. Comput. Mach. (2023)

Xia, C.S., Zhang, L.: Keep the conversation going: Fixing 162 out of 337 bugs for \$0.42 each using chatgpt. In: Proceedings of the 33rd ACM SIGSOFT International Symposium on Software Testing and Analysis (2024)

Xia, C.S., Zhang, L.: Less training, more repairing please: revisiting automated program repair via zero-shot learning. In: Proceedings of the 30th ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering, pp. 959–971 (2022)

Xiao, Y.-A., Yang, C., Wang, B., Xiong, Y.: Accelerating patch validation for program repair with interception-based execution scheduling. IEEE Trans. Softw. Eng. (2024)

Xie, H., Lei, Y., Yan, M., Yu, Y., Xia, X., Mao, X.: A universal data augmentation approach for fault localization. In: Proceedings of the 44th International Conference on Software Engineering, pp. 48–60 (2022)

Xin, Q., Reiss, S.P.: Identifying test-suite-overfitted patches through test case generation. In: Proceedings of the 26th ACM SIGSOFT International Symposium on Software Testing and Analysis, pp. 226–236 (2017)

Xin, Q., Reiss, S.P.: Leveraging syntax-related code for automated program repair. In: 2017 32nd IEEE/ACM International Conference on Automated Software Engineering (ASE), pp. 660–670 (2017). IEEE

Xin, Q., Wu, H., Tang, J., Liu, X., Reiss, S.P., Xuan, J.: Detecting, creating, repairing, and understanding indivisible multi-hunk bugs. Proceed. ACM Softw. Eng. 1(FSE), 2747–2770 (2024)

Xiong, Y., Wang, J., Yan, R., Zhang, J., Han, S., Huang, G., Zhang, L.: Precise condition synthesis for program repair. In: 2017 IEEE/ACM 39th International Conference on Software Engineering (ICSE), pp. 416–426 (2017). IEEE

Xiong, Y., Wang, B.: L2s: A framework for synthesizing the most probable program under a specification. ACM Trans. Softw. Eng. Method. (TOSEM) **31**(3), 1–45 (2022)

Xuan, J., Monperrus, M.: Learning to combine multiple ranking metrics for fault localization. In: 2014 IEEE International Conference on Software Maintenance and Evolution, pp. 191–200 (2014). IEEE

Xuan, J., Martinez, M., Demarco, F., Clement, M., Marcote, S.L., Durieux, T., Le Berre, D., Monperrus, M.: Nopol: Automatic repair of conditional statement bugs in java programs. IEEE Trans. Softw. Eng. **43**(1), 34–55 (2016)

Yang, A.Z., Le Goues, C., Martins, R., Hellendoorn, V.: Large language models for test-free fault localization. In: Proceedings of the 46th IEEE/ACM International Conference on Software Engineering, pp. 1–12 (2024)

Yang, L., Yang, C., Gao, S., Wang, W., Wang, B., Zhu, Q., Chu, X., Zhou, J., Liang, G., Wang, Q., Chen, J.: On the evaluation of large language models in unit test generation. In: Proceedings of the 39th IEEE/ACM International Conference on Automated Software Engineering (2024)

Ye, H., Martinez, M., Luo, X., Zhang, T., Monperrus, M.: Selfapr: Self-supervised program repair with test execution diagnostics. In: Proceedings of the 37th IEEE/ACM International Conference on Automated Software Engineering, pp. 1–13 (2022)

Ye, H., Martinez, M., Monperrus, M.: Neural program repair with execution-based backpropagation. In: Proceedings of the 44th International Conference on Software Engineering, pp. 1506–1518 (2022)

Yuan, Z., Liu, M., Ding, S., Wang, K., Chen, Y., Peng, X., Lou, Y.: Evaluating and improving chatgpt for unit test generation. Proceed. ACM Softw. Eng. **1**(FSE), 1703–1726 (2024)

Yuan, Y., Banzhaf, W.: Arja: Automated repair of java programs via multi-objective genetic programming. IEEE Trans. Softw. Eng. **46**(10), 1040–1067 (2018)

Zeng, M., Wu, Y., Ye, Z., Xiong, Y., Zhang, X., Zhang, L.: Fault localization via efficient probabilistic modeling of program semantics. In: Proceedings of the 44th International Conference on Software Engineering, pp. 958–969 (2022)

Zhang, Q., Fang, C., Ma, Y., Sun, W., Chen, Z.: A survey of learning-based automated program repair. ACM Trans. Softw. Eng. Method. (2023)

Zhang, Q., Fang, C., Xie, Y., Ma, Y., Sun, W., Chen, Y.Y.Z.: A systematic literature review on large language models for automated program repair. arXiv:2405.01466 (2024)

Zhang, Q., Fang, C., Zhang, T., Yu, B., Sun, W., Chen, Z.: Gamma: Revisiting template-based automated program repair via mask prediction. In: 2023 38th IEEE/ACM International Conference on Automated Software Engineering (ASE), pp. 535–547 (2023). IEEE

Zhang, M., Li, X., Zhang, L., Khurshid, S.: Boosting spectrum-based fault localization using pagerank. In: Proceedings of the 26th ACM SIGSOFT International Symposium on Software Testing and Analysis, pp. 261–272 (2017). ACM

Zhang, M., Li, Y., Li, X., Chen, L., Zhang, Y., Zhang, L., Khurshid, S.: An empirical study of boosting spectrum-based fault localization via pagerank. IEEE Trans. Softw. Eng. **47**(6), 1089–1113 (2019)

Zhu, Q., Sun, Z., Xiao, Y.-a., Zhang, W., Yuan, K., Xiong, Y., Zhang, L.: A syntax-guided edit decoder for neural program repair. In: Proceedings of the 29th ACM Joint Meeting on European Software Engineering Conference and Symposium on the Foundations of Software Engineering, pp. 341–353 (2021)

Zhu, Q., Sun, Z., Zhang, W., Xiong, Y., Zhang, L.: Tare: Type-aware neural program repair. In: 2023 IEEE/ACM 45th International Conference on Software Engineering (ICSE), pp. 1443–1455 (2023). IEEE

Zou, D., Liang, J., Xiong, Y., Ernst, M.D., Zhang, L.: An empirical study of fault localization families and their combinations. IEEE Trans. Softw. Eng. **47**(2), 332–347 (2019)

Publisher's note Springer Nature remains neutral with regard to jurisdictional claims in published maps and institutional affiliations.

Springer Nature or its licensor (e.g. a society or other partner) holds exclusive rights to this article under a publishing agreement with the author(s) or other rightsholder(s); author self-archiving of the accepted manuscript version of this article is solely governed by the terms of such publishing agreement and applicable law.

Authors and Affiliations

Bo Wang[1,2] · Ming Deng[1,2] · Mingda Chen[1,2] · Youfang Lin[1,2] · Jianyi Zhou[3] · Jie M. Zhang[4]

✉ Youfang Lin
yflin@bjtu.edu.cn

Bo Wang
wangbo_cs@bjtu.edu.cn

Ming Deng
24120317@bjtu.edu.cn

Mingda Chen
23120337@bjtu.edu.cn

Jianyi Zhou
zhoujianyi2@huawei.com

Jie M. Zhang
jie.zhang@kcl.ac.uk

[1] School of Computer Science and Technology, Beijing Jiaotong University, Beijing 100044, China

[2] Beijing Key Laboratory of Traffic Data Mining and Embodied Intelligence, Beijing 100044, China

[3] Huawei Cloud Computing Technologies Co., Ltd, Beijing, China

[4] King's College London, London, United Kingdom